Prepared for submission to JHEP

MadStree: a package for quickly evaluating arbitrary tree-level cosmological correlator

Spongebob Squarepants, Patrick Star

Abstract: `MadStree` is a Mathematica package to compute arbitrary tree-level cosmological correlators with scalar propagators numerically. It can deal with both massless and massive propagators. In this framework, a given dS tree diagram will be reduced to corresponding master integrals, and these master integrals will be solved by IBP and differential equations numerically. The usage of this package is described in detail via some examples.

Contents

1 Introduction **1**

2 Statement on AI Use **3**

3 IBP reduction and differential equations for tree-level dS integrals **4**
- 3.1 Notation: propagators, the integral family and index conventions 4
- 3.2 IBP relations from the three building blocks 8
- 3.3 d log-form differential equations 12
- 3.4 The remaining term and the contact blocks of the DE 15
- 3.5 Boundary conditions by nested blow-up 18
- 3.6 Several propagators sharing one theta 19

4 Using the package **21**
- 4.1 Installation and loading 21
- 4.2 Input formats and core functions 22
- 4.3 Numerical evaluation 28
- 4.4 Laurent-series reconstruction 31
- 4.5 Options, validation and common pitfalls 33

5 Examples **35**
- 5.1 Package examples 35
- 5.2 Application examples 36

6 Summary and outlook **44**

A Common-theta bundles and the odd-subset expansion **44**

1 Introduction

Tree-level cosmological correlators in de Sitter (dS) space are the basic objects of inflationary perturbation theory: after the in-in Feynman rules are applied, a correlator is a sum of time integrals whose integrands contain Hankel functions for massive propagators, exponentials for the external energies, and theta functions for the time-ordered propagators[1, 2]. Many analytic methods have been developed, including the cosmological bootstrap [3–18], Mellin techniques [12, 19–26], spectral decompositions [17, 27–32] and dispersive methods [33, 34]. See also [35–39] for some recent numerical studies.

A rather different route, and the one taken in this paper, is the route that precision calculations in flat-spacetime QFT rely on: Integration-By-Parts (IBP) relations [40] and the differential equations (DE) that they induce [41–44]. An IBP relation is a linear relation

inside one integral family, so it reduces every integral of the family to a small set of master integrals; differentiating the masters and reducing again gives a closed system of first-order linear DE, and when that system is written in d log form [45] the masters follow from boundary values by plain transport. In flat space this is by now a systematic and efficient standard that has been widely adopted, and the surrounding software is a large part of the reason. Generation and solving of the IBP system are automated by FIRE [46, 47] and by Kira [48–50], while the DE are solved numerically by DiffExp [51] and by the auxiliary mass flow method with its implementation AMFlow [52–55]. The auxiliary mass flow series also shows how far the underlying logic can be pushed: [56] argues that the master integrals of a family are completely fixed once the linear relations among them are given, and works this out on examples, so computing an integral becomes a problem of linear algebra plus numerical transport, with no integration input left to supply.

That combination of generality and automation is the reason to build the same machinery for dS space, and it was set up in [39, 57, 58]. Reference [57] extends IBP from the polynomial integrands of flat space to the Hankel integrands of dS, and gives universal formulas for the iterative reduction and for the d log-form DE of an arbitrary vertex integral family; these formulas govern all tree-level correlators and are the kernel of the loop-level construction of [58]. Reference [39] solves that system by power series expansion, obtains multivariate hypergeometric expressions for the homogeneous part of an arbitrary tree, and fixes the boundary data by blowing up the degenerate point produced by the multi-energy limit. Read together, these papers indicate that the IBP/DE framework has in dS the potential it has realized in flat space: reduction and DE are algorithmic, a topology is reduced and solved once, and the cost of a computation follows the topology rather than the analytic complexity of the answer.

This paper concerns the tree level, and it starts from [39, 57] rather than rederiving them. Two ingredients needed by a numerical implementation are not yet available there. The first is the massless internal propagator. Reference [57] covers the massless case only briefly and never uses the extra reduction relations that such a propagator provides, namely the compression of the four states at its two vertices into a single two-state factor shared by the whole propagator. Because a tree of physical interest usually contains several of these propagators, we build the IBP relations from the outset out of local building blocks that already include that relation, and we re-derive from them the general iterative reduction formula and the general d log-form DE formula of an arbitrary tree (Sec. 3.2, Sec. 3.3). The second ingredient is the boundary data. Reference [57] leaves the determination of the boundary conditions outside its scope, and [39] works it out for a two-vertex correlator, remarking that the same route should extend to any number of vertices. We give that extension as one construction: a strict ordering of the vertex energies freezes every theta function to 0 or 1, the factorized time integrals then return the boundary values of all masters as products of Gamma functions, and a vertex without an external leg is supplied with a private auxiliary energy inside the same system (Sec. 3.5). Because the reduced system and its boundary data are both algorithmic and the masters are obtained by numerical transport, one run delivers values at many kinematic points at a cost that does not grow with the analytic complexity of the answer, and analytic continuation becomes a transport

of the DE instead of the search for a new representation of the result.

`MadStree` is a Mathematica package that implements the construction above and numerically evaluates arbitrary tree-level cosmological correlators of scalar fields in de Sitter space. The reduction, the d log-form differential equation, the boundary data and the numerical transport are all derived by the package from the diagram the user supplies, so a tree is evaluated without writing down any intermediate formula by hand. The user input is an arbitrary tree diagram or loop diagram[1]. `MadStree` returns the master-integral values at every requested point, with their status and error estimates, exportable to CSV/JSON; in the regulator mode it returns the certified lowest power and the Laurent coefficients of the pole and finite parts. This mode is needed because whenever the vertex time powers take special values, the time integrals are defined only after regularization: the integral can sit on a pole and the time IBP becomes singular, while the boundary expansion at $\tau \to 0$ degenerates as soon as its branch exponents resonate. Introducing a common analytic regulator ε on the time powers makes the system generic, and the physical $\varepsilon \to 0$ limit is then taken from the reconstructed Laurent series.

The paper is organized as follows. In Sec. 3 we derive the formulas underlying the package. We review the definition of the vertex integral family and the derivation of their IBP relations, differential equations, and boundary conditions. We also discuss the simplification of the massless case in this section, and the reduction of several propagators that share one theta function, whose derivation is collected in App. A. In Sec. 4 we describe how to use the package: the loading procedure, the input formats and core functions, the numerical workflow and the reconstruction of the regulator series. We also summarize the principal options and common pitfalls. In Sec. 5 we collect the examples shipped with the release and present two application examples. We close with a summary and outlook in Sec. 6.

2 Statement on AI Use

The notation, the functionality and the public interfaces of this package were designed by the authors, who also indicated the overall direction and the strategy of the derivation recorded in Sec. 3; the concrete derivation was then worked out by agent programs driven by large language models. The package was implemented by those same programs on the basis of the authors' guidance and of the derivation results above; to verify the reliability of the package, the authors designed the following verification tasks, listed together with the source of the reference data in each case.

- **Single-vertex families and the massive two-vertex correlator.** The exact differential equations, the boundary coefficients and the numerically transported results are compared with the known results of [39], which are computed by an independent multivariate hypergeometric construction.

[1] For a loop diagram the package evaluates the vertex time integrals of the Schwinger-Keldysh graph; the loop-momentum integral itself is not performed.

- **The massive three-vertex tree graph.** The package results are checked against two independent sources at once: the output of an agent program that received only the previously hand-written two-vertex code and generalized it to three vertices along its own route, and the published eight-branch result of [25]. The agent output was itself first checked against that published result before being used as a reference, so the package is validated both by independently generated code and by the published numbers.

- **IBP relations and differential equations.** These are compared with the `dSIBP` package [59], which is released in the same repository as this one, `https://github.com/J7C/dSIBP-Packages`. `dSIBP` samples naive IBP seeds and solves the resulting linear reduction system, whereas the present package applies the general formulas derived in Sec. 3 directly and never re-solves an IBP system; the agreement therefore tests the closed formulas themselves.

3 IBP reduction and differential equations for tree-level dS integrals

This section derives the objects the package computes: the integral family of a tree-level dS correlator, the IBP relations that reduce the family, the d log-form differential equation (DE) of its master integrals, and the boundary data that fixes the solution. Two references underlie the construction: [57] derives the generalized IBP relations, the one-step lowering/raising recurrences and the uniform d log-form DE of an arbitrary vertex integral family, and [39] solves that system by power series expansion and fixes its boundary data by blowing up the degenerate point produced by the multi-energy limit.

Two points distinguish the present section from those references and fix the organization of the material. First, we work in the prefactor convention that the package implements, in which the Hankel block carries $z^{+\nu}$ rather than the $z^{-\nu}$ of the printed formulas of [39, 57]; relative to those papers this is the single simultaneous replacement $\nu \to -\nu$ in every occurrence, and after performing it no further translation is needed anywhere below (Sec. 3.1). Second, the one and only structural difference from [57] is the massless internal propagator: that reference never uses the extra reduction relations of a massless bulk propagator to compress the four states at the two vertices of a massless *full* propagator into a single two-state factor shared by the whole propagator, while the package does. We therefore build the IBP relations from the start out of the three local building blocks of Sec. 3.2 rather than re-deriving the single-vertex family, and we mark every quantity quoted from [57] before that extra relation is imposed by a superscript unreduced (defined in eq. (3.18)). Section 3.6 and App. A treat a further case that [57] does not discuss: several propagators that share one and the same theta function.

3.1 Notation: propagators, the integral family and index conventions

In-in Feynman rules and asymptotic behavior. Cosmological correlators are evaluated in the Schwinger–Keldysh (SK) formalism: the operator product is ordered along a closed time path, and each vertex is labelled by a contour branch, written `"+"` or `"-"`, with

one time integration $\int_{-\infty}^{0} \mathrm{d}\tau_v$ per vertex. With the positive-frequency mode function

$$u(\tau;k) = -\mathrm{i}\sqrt{\frac{\pi}{2}}\, e^{\mathrm{i}\pi(\nu/2+1/4)} H^{(d-1)/2}_{\text{Hubble}}(-\tau)^{d/2} H^{(1)}_{\nu}(-k\tau), \tag{3.1}$$

where $k = |\boldsymbol{k}|$ and $\nu^2 = d^2/4 - m^2/H^2_{\text{Hubble}}$ for a minimally coupled scalar in d spatial dimensions (ν is real or imaginary for $d = 3$), the bulk-to-bulk propagators are

$$G_>(k;\tau_1,\tau_2) \equiv u(\tau_1,k)u^*(\tau_2,k), \qquad G_<(k;\tau_1,\tau_2) \equiv u^*(\tau_1,k)u(\tau_2,k), \tag{3.2}$$

and the time-ordered combinations used by the in-in rules are

$$\begin{aligned} G_{++} &= G_>\theta(\tau_1-\tau_2) + G_<\theta(\tau_2-\tau_1), & G_{--} &= G_<\theta(\tau_1-\tau_2) + G_>\theta(\tau_2-\tau_1), \\ G_{+-} &= G_<, & G_{-+} &= G_>. \end{aligned} \tag{3.3}$$

Throughout, $\theta_{ij} \equiv \theta(\tau_i - \tau_j)$ denotes the theta function of the time difference of vertices i and j. Bulk-to-boundary propagators of massless fields, $G_\pm \propto (1 \mp \mathrm{i}k\tau)e^{\pm \mathrm{i}k\tau}$, enter only as pure-exponential blocks. The Hankel functions obey

$$\partial_\tau^2 H^{(1,2)}_\nu(-k\tau) + \frac{1}{\tau}\,\partial_\tau H^{(1,2)}_\nu(-k\tau) + \left(k^2 - \frac{\nu^2}{\tau^2}\right) H^{(1,2)}_\nu(-k\tau) = 0, \tag{3.4}$$

with the two asymptotic regimes

$$\begin{aligned} &H^{(1)}_\nu(-k\tau) = c_1(-k\tau)^\nu(1+O(\tau^2)) + c_2(-k\tau)^{-\nu}(1+O(\tau^2)), \quad \tau \to 0^-, \\ &c_1 = e^{-\mathrm{i}\pi\nu}c[\nu]\,, \quad c_2 = c[-\nu]\,, \quad c[\nu] \equiv \frac{2^{-\nu}\Gamma(-\nu)}{\mathrm{i}\pi}\,. \end{aligned} \tag{3.5}$$

and

$$\begin{aligned} H^{(1)}_\nu(-k\tau) &\sim \sqrt{\frac{2}{\pi}}(-k\tau)^{-1/2} e^{-\mathrm{i}k\tau - \mathrm{i}\pi(\nu/2+1/4)}, \\ H^{(2)}_\nu(-k\tau) &\sim \sqrt{\frac{2}{\pi}}(-k\tau)^{-1/2} e^{\mathrm{i}k\tau + \mathrm{i}\pi(\nu/2+1/4)}, \quad -k\tau \to \infty. \end{aligned} \tag{3.6}$$

The first regime fixes the boundary conditions of the time integrals; the second justifies the Wick rotation used in the numerical evaluation of Sec. 4.3.

The two-state h block. A tree-level correlator with V vertices contains one time integration per vertex, and each integrand is a product of the elementary factors

$$\left\{ e^{\mathrm{i}k\tau},\ H^{(1,2)}_\nu(-k\tau),\ \partial_\tau H^{(1,2)}_\nu(-k\tau),\ \theta(\tau_j - \tau_k) \right\}. \tag{3.7}$$

Following [39], the Hankel prefactor and the time derivative are absorbed into two states of one function system. We use the convention of the package, in which the prefactor is

the *positive* power of z,

$$\begin{aligned} h^{(s)}(\nu,0;z) &\equiv z^{+\nu}H_\nu^{(s)}(z), \\ h^{(s)}(\nu,1;z) &\equiv -\frac{1}{k}\,\partial_\tau h^{(s)}(\nu,0;z) = \partial_z h^{(s)}(\nu,0;z), \qquad z=-k\tau, \end{aligned} \tag{3.8}$$

with $s\in\{1,2\}$ the Hankel branch. Superscripts on h will always denote the branch, never a power or a label. The alternative normalization $h(\nu,0;z)=z^{-\nu}H_\nu(z)$ used in [39, 57] is related to (3.8) by $\nu\to-\nu$, because the reflection identities $H_{-\nu}^{(1)}=e^{\mathrm{i}\pi\nu}H_\nu^{(1)}$ and $H_{-\nu}^{(2)}=e^{-\mathrm{i}\pi\nu}H_\nu^{(2)}$ carry each branch into itself, multiplied only by a z-independent phase; the two phases are inverse to one another, so a normalization built from the pair of branches, such as eq. (3.43) below, is unaffected by the trade. Equivalently, with ν_{ref} the symbol printed in those two papers, $\nu_{\mathrm{ref}}=-\nu$ *everywhere at once* — in the equation of motion below, in the Wronskian, in M_1, in the contact powers, in the recurrences and in the d log potentials. For a heavy field $\nu=\mathrm{i}\mu$ with real μ this reduces to $e^{-\pi\mu}H_{-\mathrm{i}\mu}^{(2)}(z)=H_{\mathrm{i}\mu}^{(2)}(z)$, and we shall use exactly this identity in Sec. 3.4 to trade the conjugate-order basis at the two vertices of a contracted propagator in [39] for the common-order basis of the package.

Since the Hankel equation involves ν^2 only, the two normalizations satisfy different first-order systems but the same second-order one up to $\nu\to-\nu$. In the convention (3.8), with $\boldsymbol{h}=(h(\nu,0;z),h(\nu,1;z))^T$ and $P_1\equiv\frac{1}{2}(\mathbb{I}-\sigma_3)=\mathrm{diag}(0,1)$,

$$\partial_\tau\boldsymbol{h}=\left[-\mathrm{i}k\sigma_2-\frac{1-2\nu}{\tau}\,P_1\right]\boldsymbol{h}, \tag{3.9}$$

where $\sigma_{1,2,3}$ are the Pauli matrices and $\mathbb{I}$ the 2×2 identity; the equivalent dimensionless equation of motion is

$$\partial_z^2 h(\nu,0;z)+\frac{1-2\nu}{z}\,\partial_z h(\nu,0;z)+h(\nu,0;z)=0, \tag{3.10}$$

whose constant term is 1; the physical k^2 enters only through $\partial_\tau=-k\partial_z$.

The vertex integral family and its index conventions. For a vertex carrying n massive legs, the one-vertex family (an n-fold Hankel vertex integral family) reads

$$f^{(a_0)}_{\tilde{\boldsymbol{a}};\boldsymbol{s}}=\int_{-\infty}^{0}\mathrm{d}\tau\,(-\tau)^{\nu_0+a_0}e^{\mathrm{i}p_0\tau}\prod_{i=1}^{n}h^{(s_i)}(\nu_i,a_i;-k_i\tau),\quad a_0\in\mathbb{Z},\quad a_i\in\{0,1\},\quad s_i\in\{1,2\}, \tag{3.11}$$

where the indices have the following meanings, which we reuse for the whole graph:

- ν_0 is the power of the vertex time, $s_i\in\{1,2\}$ is the Hankel branch chosen at leg i by the SK type of the propagator that ends there, $a_i\in\{0,1\}$ is the *state bit* — the index of the two components of (3.8) — and a_0 is the integer *shift* of the vertex time power;
- $p_0=\varsigma k_0$ is the signed energy of the vertex (Sec. 3.2), and $k_i=|\boldsymbol{k}_i|$ the momentum modulus of leg i;

- the state bits are packed into one integer by the binary convention of [57],

$$\tilde{\boldsymbol{a}} = 1 + \sum_{i=1}^{n} a_i \, 2^{\,n-i}, \tag{3.12}$$

  so that the first bit is the most significant and the 2^n entries $\mathrm{I}_{\tilde{\boldsymbol{a}}} = \mathrm{I}_{\{\boldsymbol{a}\}}$ of the master vector are ordered exactly as the Kronecker products of the leg spaces; in the package this is `IntegerDigits[...,2,n]` and the same order is used for the whole graph.

For a graph the same three kinds of index are kept, only made global. Fix a sector s (defined below) with c_s time components and n_s^{slot} two-state slots — one for each massive propagator at each of the two vertices it joins, and one shared by the two vertices of a massless full propagator — and write a master as

$$\mathrm{I}_{\tilde{\boldsymbol{r}}} \equiv \mathrm{I}_s(\boldsymbol{n}; \boldsymbol{a}), \qquad \boldsymbol{n} \in \mathbb{Z}^{c_s}, \quad \boldsymbol{a} \in \{0,1\}^{n_s^{\mathrm{slot}}}, \tag{3.13}$$

where $\boldsymbol{n}$ assigns one integer time-power shift to each component (a component is a maximal set of vertices merged by the contacts; a single vertex is one component), $\boldsymbol{a}$ is the ordered string of state bits of all slots, packed into $\tilde{\boldsymbol{r}}$ by (3.12), and the sector s itself is recorded by a *sector key*: with the root propagators listed in input order $E = (e_1, \ldots, e_{N_E})$ and L_s^{ctr} the set of propagators contracted by contacts,

$$\mathrm{key}(s) = q_1 q_2 \cdots q_{N_E}, \qquad q_j = \begin{cases} 0, & e_j \in L_s^{\mathrm{ctr}}, \\ 1, & e_j \notin L_s^{\mathrm{ctr}}, \end{cases} \tag{3.14}$$

a fixed-length bit *string*, not a binary integer, so leading zeros are kept and the top sector is the all-1 string. Sectors are partially ordered by $t \prec s$ when t is reachable from s by at least one contact, i.e. when $L_t^{\mathrm{ctr}} \supsetneq L_s^{\mathrm{ctr}}$.

These are precisely the three arguments of the integral object of the package: the master (3.13) is `MSIntegral[sectorKey, timeShifts, stateBits]`, where `sectorKey` is the string (3.14), `timeShifts` the vector $\boldsymbol{n}$ and `stateBits` the string $\boldsymbol{a}$; the same slots are used by the returned records `"sectorKey", "stateBits", "timeShifts", "integral", "normalization"`. The object maps slot-by-slot onto the corresponding `dSIBP` object `J[sectorKey, timeShifts, stateBits]` of the parent framework [59], with the same key alphabet, the same shift dimension c_s and the same bit order, so that a MadStree master can be read as a dSIBP J and back without any re-indexing.

What the notation hides. The compactness of (3.13) is paid for by hiding six pieces of data, all of which are fixed once and for all at initialization and never re-enter the algebra:

1. the Hankel prefactor $z^{\pm\nu}$ and the normalization of the mode function (3.1): the $(-\tau)^{d/2}$ and the $z^{\pm\nu}$ powers are split between the explicit vertex power ν_0 and the definition (3.8), so no propagator carries a visible prefactor;

2. the choice of the small-τ branch and of the vanishing powers at $\tau \to 0$: both are contained in the superscript $(1, 2)$ and the order ν of $h^{(s)}$, which select between the

two terms of (3.5);

3. the sector normalization $\mathcal{N}_s$: the object the package propagates is $J_s = \mathcal{N}_s \mathrm{I}_s$, with $\mathcal{N}_s$ the product of the pinch factors of the contracted massive full propagators (Sec. 3.4) and of a per-sector constant;

4. the SK contour signs: the vertex phase sign ς_v of (3.15), the incidence ϵ_{ve} of (3.25) and the propagator sign σ_e of (3.24) are fixed by the vertex types and by the vertex order of each propagator, and enter only through the signed energies $p_{0,v}$ and the letters κ;

5. the theta chamber: at a boundary chart each theta is frozen to 0 or 1 by the strict energy order (Sec. 3.5), and inside the bulk the ordering of a bundle is carried by the single δ of Sec. 3.6;

6. the regulator: when a time power sits on a pole, a common analytic shift $\nu_0 \to \nu_0 + \varepsilon$ is understood (Sec. 4.4).

All six may be hidden for the same reason: none of them changes the number of slots, the slot order or the count of master integrals, which are the only things the IBP recurrences and the DE block structure read. Every formula below is therefore an identity of the reduced vector space, independent of the kinematics-independent constants that were absorbed.

3.2 IBP relations from the three building blocks

The relative normalization of the recurrences. Compared with [57] there is exactly one difference, already stated in Sec. 3: that paper gives each vertex of a propagator its own independent two-state factor, including the two vertices of a massless full propagator, and never imposes the extra massless relations that identify them. Everything else is its construction, translated to the $+\nu$ convention and to the $(-\tau)^{\nu_0}$ time power of (3.11). Note this last point: with $(-\tau)^A$ instead of τ^{ν_0}, the derivative of the explicit power picks up a sign, $\partial_\tau(-\tau)^A = -A(-\tau)^{A-1}$, so the one-step relation carries $-\mathrm{M}_1$ where the reference carries $+\mathrm{M}_1$; the diagonal part of the DE is unaffected but the contact block of the DE changes sign, which is why our (3.49) differs from the printed [39] form by that overall sign.

Block (i): the theta-free exponential of a vertex. Each vertex v carries exactly *one* theta-free exponential factor,

$$E_v(\tau_v) = \exp(\mathrm{i}\varsigma_v k_{0,v}\tau_v)\,, \qquad \varsigma_v = \begin{cases} -1, & v \text{ is a “+” vertex,} \\ +1, & v \text{ is a “-” vertex,} \end{cases} \qquad p_{0,v} = \varsigma_v k_{0,v}, \tag{3.15}$$

so that a `"+"` vertex gives $e^{-\mathrm{i}k_{0,v}\tau_v}$ and a `"-"` vertex gives $e^{+\mathrm{i}k_{0,v}\tau_v}$. Here $k_{0,v}$ is the *single summed* external-energy parameter of the vertex, the modulus sum of the theta-free external legs attached to v; in the package it is the input field `externalLegEnergy`.

eq. (3.1) of [57] likewise writes one k_0 per vertex, and this document never introduces per-leg or per-block exponential parameters, nor any sum over them. A theta-free propagator branch is not folded into this symbol: it keeps its own modulus, and that modulus is added to the energy of whichever component holds the vertex it ends on. After a contact has merged a set C of vertices into one component, the same scalar becomes the component energy

$$p_{0,C} = \sum_{v\in C} \varsigma_v k_{0,v} + \sum_{\substack{\text{theta-free propagators } e \\ \text{with exactly one vertex in } C}} (-\varsigma_{v_e})\, q_e, \tag{3.16}$$

where $q_e = |\boldsymbol{q}_e|$ is the momentum modulus of the propagator and v_e is the vertex of e that lies in C; a branch whose two vertices both lie in C contributes $+q_e - q_e = 0$ and is therefore already absent from this sum. The second sum collects the massless cross and external branches, which create no slot and only add their moduli, each with the contour sign of the vertex that holds them, into the same scalar. The first sum alone is $p_{0,v}$, so for a one-vertex component (3.16) reduces to $p_{0,v}$ exactly when that vertex carries no theta-free propagator branch.

Because $\dim \mathcal{E}_v = 1$, this block creates no state bit and no slot: it contributes nothing to M_1 and contributes the scalar $\mathrm{i}p_{0,v}\mathbb{I}$ to M_0.

Block (ii): the h system of a massive propagator. A massive propagator of order ν contributes, at each of the two vertices it joins, one two-state system (3.8) in the $\mathcal{H} \simeq \mathbb{C}^2$ of generators (3.9). This is the block of [57] unchanged, with $1-2\nu$ replacing the printed $2\nu+1$. Only the branch $s \in \{1,2\}$, fixed by the SK type of the propagator, differs between the two vertices it joins, and it does not change the generator $-\mathrm{i}k\sigma_2$.

Block (iii): the massless full propagator. Before the extra massless relations are imposed, a massless full propagator $e = (u_e, v_e)$ is exactly like a massive one: the vertex u_e carries two states $a \in \{0,1\}$, the vertex v_e carries two states $b \in \{0,1\}$, and the local space of this propagator is $\mathcal{H}_{e,u} \otimes \mathcal{H}_{e,v} \simeq \mathbb{C}^4$ with, in the natural order,

$$\boldsymbol{F}_e^{(4)} = (F_{00}, F_{01}, F_{10}, F_{11})^T. \tag{3.17}$$

Writing the unreduced full-graph space as the tensor product of all such two-state factors and of the one-dimensional vertex exponentials, and the corresponding vertex matrices as

$$\begin{aligned}
\mathcal{V}_s^{\text{unreduced}} &= \bigotimes_{v\in V_s} \mathcal{E}_v \otimes \bigotimes_{\text{active two-state factors } i} \mathcal{H}_i, \qquad \dim \mathcal{V}_s^{\text{unreduced}} = 2^{\#\{\text{two-state factors}\}}, \\
\mathrm{M}_{1;v}^{\text{unreduced}} &= \nu_{0,v}\mathbb{I} - \sum_{i\in I_v} (1-2\nu_i)\, \mathrm{Emb}_i(P_1), \\
\mathrm{M}_{0;v}^{\text{unreduced}} &= \mathrm{i}p_{0,v}\mathbb{I} - \mathrm{i}\sum_{i\in I_v} k_i\, \mathrm{Emb}_i(\sigma_2),
\end{aligned}$$

reproduces [57] verbatim in our convention: the superscript unreduced means "the two states at each vertex of every propagator, massive or massless, form an independent two-state factor, with the $+\nu$ prefactor (3.8) and the $(-\tau)^{\nu_0}$ time power", I_v is the set of

two-state factors contributed by the propagators that end on v, and V_s is the set of active vertices of the sector s. Here $\mathrm{Emb}_i(A)$ is the embedding of a local operator, $\mathrm{Emb}_i(A) = \mathbb{I} \otimes \cdots \otimes A \otimes \cdots \otimes \mathbb{I}$ with A in the i-th factor.

Imposing the extra massless relations is what reduces these four states at the two vertices to one shared two-state system. At the conformal value $\nu = 1/2$ the positive-prefactor block degenerates: $h^{(1,2)}(1/2, 0; z) = z^{1/2} H^{(1,2)}_{1/2}(z) \propto e^{\pm \mathrm{i}z}$, (3.10) becomes $h'' + h = 0$, and the derivative state is a branch eigenstate of the zeroth-order state,

$$h^{(1)}(1/2, 1) = +\mathrm{i} h^{(1)}(1/2, 0), \qquad h^{(2)}(1/2, 1) = -\mathrm{i} h^{(2)}(1/2, 0), \tag{3.18}$$

so the two states are the exponentials $e^{\pm \mathrm{i}z}$. More generally every half-integer order is a polynomial times an exponential and reduces to $\nu = 1/2$. Applied to the four states of a massless full propagator of momentum modulus q, (3.18) turns the two extra relations into identities among explicit integrands. Writing them out, the four entries of (3.17) are

$$F_{ab} = \int_{-\infty}^{0} \mathrm{d}\tau_u \int_{-\infty}^{0} \mathrm{d}\tau_v \; \Phi_e \, h^{(1)}\big(\tfrac{1}{2}, a; -q\tau_u\big) \, h^{(2)}\big(\tfrac{1}{2}, b; -q\tau_v\big) \,, \qquad a, b \in \{0, 1\}, \tag{3.19}$$

where Φ_e collects every other factor of the integrand: the vertex exponentials of u and v, the theta of this propagator, and the factors of all the remaining propagators. The first branch sits at u_e and the second at v_e for G_{++}; for G_{--} the two branches are exchanged. Each differentiated state is one factor $\pm\mathrm{i}$ times the undifferentiated one at the same vertex by (3.18), so

$$F_{10} = \mathrm{i} F_{00} = -F_{01}, \qquad F_{11} = \mathrm{i}(-\mathrm{i}) F_{00} = F_{00} \qquad (G_{++}), \tag{3.20}$$

and for G_{--} the two factors i and $-\mathrm{i}$ are interchanged, which leaves the same two identities $F_{10} = -F_{01}$ and $F_{11} = F_{00}$. Only two of the four states are therefore independent. Choosing the shared two-state $\boldsymbol{g}_e = (F_{00}, F_{10})^T \in \mathcal{M}_e \simeq \mathbb{C}^2$, the constant embedding and projection are

$$\boldsymbol{F}^{(4)}_e = S_e \boldsymbol{g}_e, \qquad S_e = \begin{pmatrix} 1 & 0 \\ 0 & -1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad P_e = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \qquad P_e S_e = \mathbb{I}_2, \tag{3.21}$$

and every four-dimensional operator that preserves the relation subspace (3.20) reduces to $P_e O S_e$. In particular

$$P_e(\sigma_2 \otimes \mathbb{I}_2) S_e = \sigma_2, \qquad P_e(\mathbb{I}_2 \otimes \sigma_2) S_e = -\sigma_2, \tag{3.22}$$

so the two vertices joined by a massless full propagator act with *the same* σ_2 on *the same* two-dimensional slot, with opposite signs. This is the point of the reduction: the shared slot carries one bit r_e that enters the state string of both vertices joined by e and cannot be duplicated into two independent two-state factors. Consequently the full-graph tensor

structure survives, but the vertex-by-vertex factorization $\mathcal{V}_s \stackrel{?}{=} \bigotimes_{v\in V_s} \mathcal{V}_v$ fails, because $\mathcal{M}_e$ cannot belong to both u_e and v_e at once. The reduced space of a sector is

$$\mathcal{V}_s = \bigotimes_{v\in V_s} \mathcal{E}_v \otimes \bigotimes_{i\in H(s)} \mathcal{H}_i \otimes \bigotimes_{e\in E_m(s)} \mathcal{M}_e, \qquad \dim \mathcal{V}_s = 2^{|H(s)|+|E_m(s)|}, \tag{3.23}$$

where $H(s)$ is the set of two-state factors that the massive propagators of s still contribute and $E_m(s)$ the set of its uncontracted massless full propagators.

The contact vector of a theta derivative is $\boldsymbol{\rho}_e^{(4)} = (0,-1,+1,0)^T$, and (3.21) identifies it exactly as $\boldsymbol{\rho}_e^{(4)} = S_e|1\rangle_e$ with the fixed column $|1\rangle_e = (0,1)^T$; the contact selector of the shared slot therefore needs no further computation. What remains to fix is the numerical factor which this derivative carries at each of the two vertices. With $\sigma_e = +1$ for G_{++} and $\sigma_e = -1$ for G_{--}, the two vertices u_e and v_e, taken in that order, carry the *contact weights*

$$w_e^{(u_e)} = +2\mathrm{i}\sigma_e, \qquad w_e^{(v_e)} = -2\mathrm{i}\sigma_e, \tag{3.24}$$

in the h-state basis of (3.8). They are the numbers that multiply the local selector of the contracted propagator in the injection matrix $C_{st}^{(v,1)}$ of a contact event, which is the object built in (3.45) and carried into the contact block $\Omega_{st}^{\mathrm{contact}}$ of (3.49); the phase χ_v of (3.50) records how the pair (3.24) changes once the package basis is used. The package quotient instead factors the derivative phase of each vertex out of its state, which is the constant basis change $D_{\sigma_e} = \mathrm{diag}(1, \mathrm{i}\sigma_e)$; it turns $-\mathrm{i}q_e\sigma_2$ into $\pm\mathrm{i}\sigma_e q_e\sigma_1$ and (3.24) into the pair $(-2,+2)$, and it is diagonalized by the Hadamard matrix $\frac{1}{\sqrt{2}}\left(\begin{smallmatrix}1 & 1\\ 1 & -1\end{smallmatrix}\right)$ in place of T. Both bases give the same set of energy letters, since D_{σ_e} only relabels the shared bit.

Local generators and the full-graph $\mathbf{M_1}$, $\mathbf{M_0}$. For each massless full propagator, whose two vertices we order as (u_e, v_e), define the incidence

$$\epsilon_{ve} = \begin{cases} +1, & v = u_e, \\ -1, & v = v_e, \\ 0, & v \notin e, \end{cases} \tag{3.25}$$

and let $H_v(s)$ be the set of two-state factors that the massive propagators of s contribute at v. Each building block then contributes to the vertex v the pair of atoms

$$\Delta\mathrm{M}_1^{(E_v)} = 0, \qquad \Delta\mathrm{M}_0^{(E_v)} = \mathrm{i}p_{0,v}\mathbb{I}, \qquad \text{the exponential of the vertex } v, \tag{3.26}$$

$$\Delta\mathrm{M}_1^{(h_i)} = -(1-2\nu_i)P_1, \quad \Delta\mathrm{M}_0^{(h_i)} = -\mathrm{i}k_i\sigma_2, \qquad \text{a massive propagator of order } \nu_i \text{ at } v, \tag{3.27}$$

$$\Delta\mathrm{M}_1^{(\mathcal{M}_e)} = 0, \qquad \Delta\mathrm{M}_0^{(\mathcal{M}_e)} = -\mathrm{i}\epsilon_{ve}q_e\sigma_2, \quad \text{a massless propagator } (\nu = \tfrac{1}{2}) \text{ at } v. \tag{3.28}$$

The second row is one of the two states that a massive propagator contributes at the vertex v it ends on; the third is the internal propagator whose ν is a half-integer, i.e. a massless

one. The massless row is written once for each of the two vertices: in (3.28) the factor ϵ_{ve} carries the sign with which the same operator $\mathrm{Emb}_e(\sigma_2)$ appears in the rows of u_e and of v_e. A massless internal propagator does not enter M_1 at all: its $\nu = 1/2$ makes $1 - 2\nu$ vanish identically in (3.27). The two complete matrices of a vertex v in a sector s are then the sum of these atoms, each embedded in the slot it acts on, together with the $\nu_{0,v}\mathbb{I}_s$ of the vertex time power,

$$\begin{aligned}
\mathrm{M}_{1;v,s}(\nu_{0,v}) &= \nu_{0,v}\mathbb{I}_s + \mathrm{Emb}_{\mathcal{E}_v}\Big(\Delta\mathrm{M}_1^{(E_v)}\Big) + \sum_{i\in H_v(s)} \mathrm{Emb}_i\Big(\Delta\mathrm{M}_1^{(h_i)}\Big) \\
&\quad + \sum_{e\in E_m(s)} \mathrm{Emb}_e\Big(\Delta\mathrm{M}_1^{(\mathcal{M}_e)}\Big) \\
&= \nu_{0,v}\mathbb{I}_s - \sum_{i\in H_v(s)} (1-2\nu_i)\,\mathrm{Emb}_i(P_1), \qquad (3.29) \\
\mathrm{M}_{0;v,s} &= \mathrm{Emb}_{\mathcal{E}_v}\Big(\Delta\mathrm{M}_0^{(E_v)}\Big) + \sum_{i\in H_v(s)} \mathrm{Emb}_i\Big(\Delta\mathrm{M}_0^{(h_i)}\Big) + \sum_{e\in E_m(s)} \mathrm{Emb}_e\Big(\Delta\mathrm{M}_0^{(\mathcal{M}_e)}\Big) \\
&= \mathrm{i}p_{0,v}\mathbb{I}_s + \sum_{i\in H_v(s)} \mathrm{Emb}_i(-\mathrm{i}k_i\sigma_2) \\
&\quad + \sum_{e\in E_m(s)} \mathrm{Emb}_e(-\mathrm{i}\epsilon_{ve}q_e\sigma_2). \qquad (3.30)
\end{aligned}$$

The first line of (3.29) and the first line of (3.30) are just the atoms (3.26)–(3.28) collected at v; the second line of each is what they give once those atoms are inserted. The per-vertex form is what (3.18) already gives, and the reduction only replaces the two two-state factors at the two vertices of a massless propagator by one shared factor: ∂_{τ_v} acts on the blocks belonging to v and on nothing else, so the vertex generators remain Kronecker *sums* on the common space (3.23) and different vertices meet only on a shared massless slot, where they carry the same σ_2 and opposite ϵ_{ve}.

3.3 d log-form differential equations

Master choice and the regular block. The master integrals of a sector are the entries of (3.11) with vanishing time-power shift, $\mathrm{I}_{\tilde{\boldsymbol{a}}} = f^{(0)}_{\tilde{\boldsymbol{a}};\boldsymbol{s}}$, collected into the ordered vector $\boldsymbol{F}_s(\mathbf{0})$; the collection over all sectors reachable by contacts is $\{\boldsymbol{F}_s(\mathbf{0})\}$. They obey a d log-form DE,

$$\begin{aligned}
\mathrm{d}\boldsymbol{F} &= (\mathrm{d}\Omega)\cdot\boldsymbol{F} = \sum_x \Omega_x \cdot \boldsymbol{F}\,\mathrm{d}x\,, \\
\Omega^{\mathrm{reg}}_{ss} &= \Omega_{\mathrm{ex},s} - \mathrm{i}\sum_{v\in V_s} U_s^{-1}\cdot\widetilde{\Omega}_{0;v,s}\cdot U_s\cdot \mathrm{M}_{1;v,s}(\nu_{0,v}+1)\,, \qquad (3.31)
\end{aligned}$$

where x runs over the independent kinematic variables and $\Omega_x = \partial_x\Omega$. The derivation is that of [57]: differentiate the master vector, reduce the differentiated integrals back onto the masters by applying the one-step recurrence (3.37) iteratively, and read off the connection. The massless reduction changes nothing in that route except the two matrices the answer

is expressed through: $\mathrm{M}_{1;v,s}$ and $\mathrm{M}_{0;v,s}$ are now built on the reduced space (3.23), and the diagonalizer of [57] is replaced by the U_s of (3.35), which carries an extra factor on every shared slot. Because the reduced generators still commute slot by slot, that one U_s diagonalizes every vertex matrix at once, so (3.31) follows from the printed formula of [57] by this single replacement. The two ingredients are diagonal matrices indexed by the state bits,

$$\begin{aligned}\left(\widetilde{\Omega}_{0;v,s}\right)_{\boldsymbol{ba}} &\equiv \begin{cases} -\mathrm{i}\log\kappa_{v,s;\boldsymbol{a}}, & \boldsymbol{b}=\boldsymbol{a} \\ 0, & \boldsymbol{b}\neq\boldsymbol{a} \end{cases}, \\ (\Omega_{\mathrm{ex,s}})_{\boldsymbol{ba}} &\equiv \begin{cases} -\sum_{i\in H(s)} a_i(1-2\nu_i)\log k_i, & \boldsymbol{b}=\boldsymbol{a} \\ 0, & \boldsymbol{b}\neq\boldsymbol{a} \end{cases}, \end{aligned} \tag{3.32}$$

with the energy letters

$$\kappa_{v,s;\boldsymbol{a}} = p_{0,v} + \sum_{i\in H_v(s)} (2a_i-1)k_i + \sum_{e\in E_m(s)} \epsilon_{ve}(2r_e-1)q_e, \tag{3.33}$$

in which $\boldsymbol{a}$ abbreviates the full bit string, massive bits a_i and shared bits r_e together. The same r_e enters the letters of u_e and v_e with opposite signs: this is the whole regular content of the correlation carried by one shared propagator, and it is why the letter set of a graph with shared slots is still a set of linear forms in the momenta. Only the states of massive propagators contribute to Ω_{ex}, because a massless slot has $1-2\nu=0$.

$\mathrm{M}_{1;v,s}$ is diagonal in the original basis and all $\mathrm{M}_{0;v,s}$ are made of σ_2 (or, in the package quotient, of σ_1) on individual slots, so they commute and are simultaneously diagonalized by one constant matrix, the Kronecker product of the massive diagonalizer T with the diagonalizer of the shared slots,

$$(\mathrm{M}_{1;v,s}[\nu_{0,v},\boldsymbol{\nu}])_{\boldsymbol{ba}} = \begin{cases} \nu_{0,v} - \sum_i a_i(1-2\nu_i), & \boldsymbol{b}=\boldsymbol{a} \\ 0, & \boldsymbol{b}\neq\boldsymbol{a} \end{cases}, \tag{3.34}$$

$$(\mathrm{T}_n)_{\boldsymbol{ba}} = \prod_{i=1}^{n} \mathrm{T}_{b_i a_i}, \qquad \mathrm{T} = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & -\mathrm{i} \\ -\mathrm{i} & 1 \end{pmatrix},$$

$$U_s = \bigotimes_{i\in H(s)} \mathrm{T}_i \otimes \bigotimes_{e\in E_m(s)} \mathrm{T}_e, \qquad \widetilde{\mathrm{M}}_{0;v,s} = U_s \mathrm{M}_{0;v,s} U_s^{-1} = \mathrm{diag}(\mathrm{i}\kappa_{v,s;\boldsymbol{a}}). \tag{3.35}$$

Here T diagonalizes σ_2 ($\mathrm{T}\sigma_2\mathrm{T}^{-1}=\sigma_3$); on the shared massless slots of the package quotient the same factor is instead the Hadamard matrix, which diagonalizes σ_1, and the two choices give the same set of letters (3.33) because D_{σ_e} only relabels the shared bit. The condition for one constant U_s to diagonalize every vertex matrix slot by slot is that all the Pauli directions arriving on a given slot mutually commute, which holds here because every massive slot receives only σ_2 and every shared slot only σ_2 (or, in the quotient, only σ_1). For a single vertex all of whose legs are massive the sums reduce to one term with $U=\mathrm{T}_n$ and (3.31) is the uniform formula of [39], in our convention.

No dense $2^N \times 2^N$ inversion ever appears: inverting $\mathrm{M}_{0;v,s}$ means

$U_s^{-1}\,\mathrm{diag}(1/(\mathrm{i}\kappa_{v,s;\boldsymbol{a}}))U_s$, i.e. the letter-by-letter reciprocal of the scalar letters (3.35), and M_1^{-1} is the reciprocal of its diagonal entries. Changing to user or Wick coordinates does not rederive the DE, it only pulls back the one-form, and changing basis to the package quotient is the constant similarity transformation generated by the D_{σ_e}.

What the extra relations change. Three statements isolate the effect of the massless reduction on the DE. (1) A massless shared slot contributes nothing to Ω_{ex} and nothing to M_1, so the diagonal logarithmic structure in the orders ν_i is exactly the massive one. (2) It contributes one state bit r_e to the letters of both vertices with opposite incidence (3.33), so the two vertices share a bit while their recurrences stay separate closed-form matrices. (3) The theta derivative of a full propagator, massive or massless, is not a regular operator: it injects the parent relation into strictly lower sectors, which produces the off-diagonal blocks (3.49); and the time power of the merged component counts vertex merges, not contracted propagators, so the base power of a component C is

$$A_{C,s} = \sum_{v\in C} \nu_{0,v} + \sum_{e\in E_s(C)} \varrho_e + |C| - 1, \tag{3.36}$$

where $E_s(C)$ is the set of propagators contracted into C, and ϱ_e is the raw contact power of the propagator — $\varrho_e = 2\nu_e - 1$ for a massive full propagator and $\varrho_e = 0$ for a massless one, as will follow from (3.44) and (3.20). This ϱ_e is a fixed number of the propagator, not to be confused with the shared state bit r_e of (3.33). The last term is the number of vertex merges that actually occur, which is why a simultaneous contact of k shared-theta propagators contributes once and not k times (App. A).

Iterative reduction. The recurrences are per vertex and per time component: every component C of a sector has its own integer shift n_C, and one step changes exactly one of them. Since a merged component is integrated over a single time, the power and the energy that a step reads are those of the component, not of the individual vertex: the relations below are written with $\nu_{0,v}$ and $p_{0,v}$ standing for $A_{C,s}$ of (3.36) and $p_{0,C}$ of (3.16) whenever v has been merged into a larger C, and they reduce to the vertex quantities exactly when $C = \{v\}$. The same reading applies to (3.31)–(3.35). Writing $\boldsymbol{e}_C$ for the unit vector of the component that contains v, the complete one-step IBP at v is

$$-\mathrm{M}_{1;v,s}(A_{C,s} + n_C)\,\boldsymbol{F}_s(\boldsymbol{n} - \boldsymbol{e}_C) + \mathrm{M}_{0;v,s}\,\boldsymbol{F}_s(\boldsymbol{n}) + \mathcal{R}_{v,s}(\boldsymbol{n}) = 0, \tag{3.37}$$

where the minus in front of M_1 is the $(-\tau)^A$ derivative and $\mathcal{R}_{v,s}$ collects the theta contact terms of the propagators incident to v (next subsection). Solving (3.37) for the vector at $\boldsymbol{n} - \boldsymbol{e}_C$, and the same relation taken one step higher for the one at $\boldsymbol{n} + \boldsymbol{e}_C$, gives the two directions

$$A_{-,v,s}(\boldsymbol{n}) = \mathrm{M}_{1;v,s}(A_{C,s} + n_C)^{-1}\mathrm{M}_{0;v,s}, \tag{3.38}$$

$$A_{+,v,s}(\boldsymbol{n}) = U_s^{-1}\widetilde{\mathrm{M}}_{0;v,s}^{-1}U_s\,\mathrm{M}_{1;v,s}(A_{C,s} + n_C + 1), \tag{3.39}$$

so $\boldsymbol{F}_s(\boldsymbol{n} \pm \boldsymbol{e}_C)$ is expressed through $\boldsymbol{F}_s(\boldsymbol{n})$ by two diagonal inverses only, up to $\mathcal{R}$: the lowering step carries $+\mathrm{M}^{-1}_{1;v,s}\mathcal{R}_{v,s}(\boldsymbol{n})$ and the raising step $-\mathrm{M}^{-1}_{0;v,s}\mathcal{R}_{v,s}(\boldsymbol{n}+\boldsymbol{e}_C)$. Note that both signs are the opposite of the printed [57] ones: there the explicit power is τ^A and the one-step relation carries $+\mathrm{M}_1$, whereas (3.37) carries $-\mathrm{M}_1$ for the $(-\tau)^A$ power of (3.11). The lowering step is singular when an eigenvalue of M_1 vanishes, $A_{C,s}+n_C-\sum_i a_i(1-2\nu_i) = 0$; this is a property of the state and the time powers, not of the kinematics, and it is the layer the regulator of Sec. 4.4 moves off. The raising step is singular on the kinematic hypersurface $\kappa_{v,s;\boldsymbol{a}} = 0$, i.e. when the momenta become collinear and split into two opposite groups; there the DE has its poles, and the letter count is exactly the number of such hypersurfaces.

3.4 The remaining term and the contact blocks of the DE

Two different remaining terms. The contact term $\mathcal{R}_{v,s}$ in (3.37) depends on the shift at which it is evaluated, and the two directions must not be confused. Reducing the lower time powers of the child first gives the *lowering* version

$$\mathcal{R}^{(0)}_{v,s} \equiv \mathcal{R}_{v,s}(\boldsymbol{n}), \qquad \text{child reached at shift } -1, \tag{3.40}$$

while the contact that appears in the raising row (3.39) is

$$\mathcal{R}^{(1)}_{v,s} \equiv \mathcal{R}_{v,s}(\boldsymbol{n}+\boldsymbol{e}_C), \qquad \text{child reached at shift } 0. \tag{3.41}$$

The reason is the base-power rule (3.36): the raw contact integrand carries one factor $(-\tau)^{\varrho_e}$ too few to be a master of the child at vanishing shift, so $\mathcal{R}^{(0)}$ lands on child shift -1 and only $\mathcal{R}^{(1)}$ — obtained by raising the parent power by one, which raises the merged component by one — hits the child master itself. Since the DE differentiates the masters $\boldsymbol{F}_s(\boldsymbol{0})$ and therefore uses the raising row, *the off-diagonal blocks of the DE must be built from* $\mathcal{R}^{(1)}$; substituting $\mathcal{R}^{(0)}$ gives the wrong sign and a non-closing expression.

The two-vertex massive contraction. Take two vertices u, v joined by one massive full propagator with order ν at each of its two vertices, momentum modulus k, the legs i at u and j at v; the two other legs are collected in $\boldsymbol{a}_{\hat{\imath}}$ and $\boldsymbol{b}_{\hat{\jmath}}$. The SK combination of the two h systems at u and v is, with the branch convention (3.3),

$$\begin{aligned} &h(\nu_i, a_i, -k\tau_u)\,\theta^{(i,j)}_{uv}\,h(\nu_j, b_j, -k\tau_v) \\ &\qquad \equiv h^{(1)}(\nu_i, a_i, -k\tau_u)\,\theta_{uv}\,h^{(2)}(\nu_j, b_j, -k\tau_v) + h^{(2)}(\nu_i, a_i, -k\tau_u)\,\theta_{vu}\,h^{(1)}(\nu_j, b_j, -k\tau_v), \\ &\nu_i = \nu_j \equiv \nu, \qquad k_i = k_j \equiv k, \end{aligned}$$

where the relative sign between the two terms is that of G_{++}, and G_{--} exchanges the branches. Differentiating with respect to τ_u produces, besides the regular generators, $\partial_{\tau_u}\theta_{uv} = \delta(\tau_u - \tau_v)$ and $\partial_{\tau_u}\theta_{vu} = -\delta(\tau_u - \tau_v)$, so the delta sets the two times equal, $\tau_u = \tau_v \equiv \tau$, and leaves the antisymmetric difference

$$\delta(\tau_u - \tau_v)\Big[h^{(1)}(\nu, a_i; z)\,h^{(2)}(\nu, b_j; z) - h^{(2)}(\nu, a_i; z)\,h^{(1)}(\nu, b_j; z)\Big], \qquad z = -k\tau. \tag{3.42}$$

If $a_i = b_j$ the bracket is zero identically, since the two summands then coincide; only complementary states survive, which is the origin of the selector $\delta_{a_i,1-b_j}$. For $a_i = 0, b_j = 1$ the bracket is the Wronskian of the two branches of the h system, and with (3.8) and $H_\nu^{(1)}\partial_z H_\nu^{(2)} - \partial_z H_\nu^{(1)} H_\nu^{(2)} = -4\mathrm{i}/(\pi z)$,

$$\begin{aligned} h^{(1)}(\nu,0;z)\,h^{(2)}(\nu,1;z) - h^{(1)}(\nu,1;z)\,h^{(2)}(\nu,0;z) &= z^{2\nu}\left(H_\nu^{(1)}\partial_z H_\nu^{(2)} - \partial_z H_\nu^{(1)} H_\nu^{(2)}\right) \\ &= -\frac{4\mathrm{i}}{\pi} z^{2\nu-1}. \end{aligned} \tag{3.43}$$

For $a_i = 1, b_j = 0$ the bracket is the negative of (3.43), so both cases are collected by the factor $(-1)^{a_i+1}$. Finally $z^{2\nu-1} = k^{2\nu-1}(-\tau)^{2\nu-1}$ on the physical branch, and the surviving $(-\tau)^{2\nu-1}$ multiplies the two explicit vertex powers, which gives the raw contact power $\varrho_e = 2\nu - 1$ quoted after (3.36). The remaining term of the lowering row is therefore

$$\mathcal{R}^{(0)}_{\tilde{\boldsymbol{r}};\sigma_e} = -\sigma_e\,\delta_{a_i,1-b_j}\,(-1)^{a_i+1}\,\frac{4\mathrm{i}}{\pi}\,k^{\,2\nu-1}\;\boldsymbol{f}^{(-1)}_{\tilde{\boldsymbol{r}}}, \qquad \tilde{\boldsymbol{r}} = \boldsymbol{a}_{\hat{\imath}}\boldsymbol{b}_{\hat{\jmath}}, \tag{3.44}$$

where the sign σ_e is the propagator sign of (3.24), and $\boldsymbol{f}^{(-1)}_{\tilde{\boldsymbol{r}}}$ is the one-component child master of the merged vertex with state bits $\tilde{\boldsymbol{r}}$ and base power $\nu_{0,u} + \nu_{0,v} + 2\nu - 1$, i.e. child shift -1 with respect to (3.36). The version needed by the DE is (3.44) with $\boldsymbol{f}^{(-1)} \to \boldsymbol{f}^{(0)}$, whose base power is $\nu_{0,u} + \nu_{0,v} + 2\nu$ and which is an actual master of the child sector.

Two comments on the relation to [39] are in order. First, the printed form of this equation in that reference carries an extra Boltzmann factor $e^{\pi\,\mathrm{Im}\,\nu}$ and the power $k^{-2\nu-1}$: it is written in the negative-prefactor convention, which replaces $k^{2\nu-1} \to k^{-2\nu-1}$, and its basis of states at the two vertices of the contracted propagator uses conjugate Hankel orders. Our basis (3.8) uses the same order ν at both vertices, and the two bases are related by the constant identity $e^{-\pi\mu}H^{(2)}_{-\mathrm{i}\mu} = H^{(2)}_{\mathrm{i}\mu}$ for $\nu = \mathrm{i}\mu$, so that in the common-order basis the explicit Boltzmann factor is absorbed into this change of basis and does not appear in (3.44). Second, the same constant $-\sigma_e\frac{4\mathrm{i}}{\pi}k^{2\nu-1}$ is what the package assigns to the sector normalization $\mathcal{N}_s$ of the child as its pinch factor; because the contact blocks of the normalized masters $J_s = \mathcal{N}_s \mathrm{I}_s$ carry the ratio $\mathcal{N}_s/\mathcal{N}_t$, that constant cancels from the normalized DE, and the block (3.49) below has no residual pinch prefactor.

Injection structure and the contact block. For a general graph the contact part of the row at v is a sum over lower sectors,

$$\mathcal{R}^{(1)}_{v,s}(\boldsymbol{e}_C) = \sum_{t \prec s} C^{(v,1)}_{st}\,\boldsymbol{F}_t(\boldsymbol{0}), \tag{3.45}$$

where $C^{(v,1)}_{st}$ is the constant injection matrix of the event that takes s to t when acted on by ∂_{τ_v}: it is the tensor product of the local contact selectors of the selected propagators — $|1\rangle_e$ on a massless shared slot, the complementary-bit selector of (3.44) on a pair of massive-propagator states — times the weights (3.24), embedded in the global bit order with all spectator bits preserved by Kronecker deltas, and times the odd-subset coefficient of Sec. 3.6. By the choice of base power, the child appears at vanishing shift and the

right-hand side of (3.45) is already a master vector; only if a generalized event lands on a non-zero child shift does the same recurrence have to be applied inside the child.

Now differentiate the masters with respect to the component energy. From $\partial_{p_{0,C}} \mathrm{I}^{(A)} = -\mathrm{i}\,\mathrm{I}^{(A+1)}$, which is where the $(-\tau)^A$ power shows its sign, the raising row of (3.37) at $\boldsymbol{n} = \mathbf{0}$,

$$\mathrm{M}_{0;v,s}\,\boldsymbol{F}_s(\boldsymbol{e}_C) = \mathrm{M}_{1;v,s}(\nu_{0,v}+1)\,\boldsymbol{F}_s(\mathbf{0}) - \mathcal{R}^{(1)}_{v,s}(\boldsymbol{e}_C), \tag{3.46}$$

gives

$$\partial_{p_{0,C}} \boldsymbol{F}_s(\mathbf{0}) = -\mathrm{i}\,\mathrm{M}^{-1}_{0;v,s}\mathrm{M}_{1;v,s}(\nu_{0,v}+1)\,\boldsymbol{F}_s(\mathbf{0}) + \mathrm{i}\,\mathrm{M}^{-1}_{0;v,s}\mathcal{R}^{(1)}_{v,s}(\boldsymbol{e}_C). \tag{3.47}$$

Since $\widetilde{\mathrm{M}}_{0;v,s} = \mathrm{diag}(\mathrm{i}\kappa_{v,s;\boldsymbol{a}})$ and $\partial_{p_{0,C}}\kappa_{v,s;\boldsymbol{a}} = \delta_{vC}$, the diagonalization of (3.35) is exactly

$$\mathrm{M}^{-1}_{0;v,s} = U_s^{-1}\,\mathrm{diag}\!\left(\frac{1}{\mathrm{i}\kappa_{v,s;\boldsymbol{a}}}\right) U_s = \partial_{p_{0,C}}\big(U_s^{-1}\widetilde{\Omega}_{0;v,s}U_s\big) \equiv \partial_{p_{0,C}}\mathcal{K}_{v,s}, \tag{3.48}$$

with $\mathcal{K}_{v,s} \equiv U_s^{-1}\widetilde{\Omega}_{0;v,s}U_s$. The first term of (3.47) is therefore the $p_{0,C}$ derivative of the regular potential in (3.31), and the second one, with (3.45), is the derivative of a purely off-diagonal potential. Reading off the coefficient of $\boldsymbol{F}_t(\mathbf{0})$,

$$\Omega^{\mathrm{contact}}_{st} = +\mathrm{i}\sum_{v\in V_s} \chi_v(E_{st})\,\mathcal{K}_{v,s}\,C^{(v,1)}_{st}, \qquad t \prec s, \tag{3.49}$$

where E_{st} is the set of propagators actually selected by that simultaneous event and the phase χ_v separates the two ways a shared slot enters:

$$\chi_v(E_{st}) = \begin{cases} \varsigma_v, & \text{if } E_{st} \text{ contains no massless full propagator,} \\ (-1)^{N_0(E_{st})}, & \text{otherwise,} \end{cases} \tag{3.50}$$

where $N_0(E_{st}) = \#\{e \in E_{st} : e \text{ massless full}\}$ is the number of massless full propagators contracted by the event. For a purely massive event the only sign left in the derivative is that of the exponential quotient of the differentiated vertex, which is ς_v; a pure-massive component after a contact must have a single ς, and a mixture of positive and negative phases in one component is not a case the formula describes. For an event that contracts massless full propagators, the $-2, +2$ contact weights of the package quotient relative to the $\pm 2\mathrm{i}\sigma_e$ of (3.24) contribute one sign per shared slot, hence $(-1)^{N_0}$. All remaining sign correlations are already inside $C^{(v,1)}_{st}$. Differentiating the defining integral of a single pure-massless full propagator with respect to both vertex energies and substituting (3.49) makes the DE residual vanish component by component, which is the independent check the package runs on this block.

Ordering the sectors by the number of remaining full propagators, the complete connection is block triangular,

$$\mathrm{d}\begin{pmatrix} \boldsymbol{F}_{s_0} \\ \boldsymbol{F}_{s_1} \\ \vdots \end{pmatrix} = \mathrm{d}\begin{pmatrix} \Omega_{s_0 s_0} & \Omega_{s_0 s_1} & \cdots \\ 0 & \Omega_{s_1 s_1} & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}\begin{pmatrix} \boldsymbol{F}_{s_0} \\ \boldsymbol{F}_{s_1} \\ \vdots \end{pmatrix}, \tag{3.51}$$

so the DE is solved sector by sector, top sector first: contact only ever points from a parent to a strictly lower sector, never the reverse.

3.5 Boundary conditions by nested blow-up

The last ingredient is the boundary condition at the large-energy limit. The factor $e^{\mathrm{i}p_{0,v}\tau_v}$ is suppressed when $k_{0,v} \to \infty$ after a Wick rotation, so in this limit only the region $\tau \to 0$ contributes. Taken at face value this is one multivariate limit in which all energies diverge together, and the point it approaches is in general a multivariate degenerate singularity of the differential equation: several singular divisors meet there, so a direct series expansion about that point is not legitimate. The expansion is made legitimate by blowing the point up first; the nested coordinates introduced below are exactly the blow-up charts that replace the degenerate point by a chain of one-energy limits. This continues the route taken in [39], where the boundary condition of the multivariate hypergeometric system is fixed by the same resolution of its degenerate point. In a V-vertex correlator there are V exponential factors with energies $k_{0,v}$; assuming the ordering

$$k_{\sigma(1)} \gg k_{\sigma(2)} \gg \cdots \gg k_{\sigma(V)} \gg 1, \tag{3.52}$$

with a permutation σ, we define the nested coordinates

$$x_j = \frac{k_{\sigma(j+1)}}{k_{\sigma(j)}} \quad (1 \le j < V), \qquad x_V = \frac{1}{k_{\sigma(V)}}, \tag{3.53}$$

so that

$$\frac{1}{k_{\sigma(j)}} = \prod_{r=j}^{V} x_r, \qquad x_1, \ldots, x_V \longrightarrow 0. \tag{3.54}$$

The exponential localization scales satisfy

$$|\tau_{\sigma(1)}| \ll |\tau_{\sigma(2)}| \ll \cdots \ll |\tau_{\sigma(V)}| \ll 1. \tag{3.55}$$

Since all τ approach zero from the negative side, large k corresponds to small $|\tau|$; hence $k_u \gg k_v$ implies $\tau_u > \tau_v$, and therefore

$$\theta(\tau_u - \tau_v) \longrightarrow 1, \qquad \theta(\tau_v - \tau_u) \longrightarrow 0. \tag{3.56}$$

For G_{++} or G_{--}, (3.56) must be applied to every explicit theta in the propagator expansion according to the order of its argument, and for a bundle of propagators sharing one theta it is applied to that single theta, which is the boundary counterpart of Sec. 3.6: the bundle contributes one frozen 0 or 1, not one per propagator. All thetas in an arbitrary time graph therefore become 0 or 1. If a term requires a cyclic inequality, for example

$$\theta(\tau_1 - \tau_2)\theta(\tau_2 - \tau_3)\theta(\tau_3 - \tau_1), \tag{3.57}$$

then it is automatically zero in every strict-total-order chart, instead of producing a new boundary region.

Applying the analysis above fixes every theta to 0 or 1 according to the strict energy order, so all master integrals — in the top sector as well as in every subsector — factorize into a product of one-dimensional time integrals, one per *time component* of the sector: once the thetas are frozen, a component is integrated over its single time, and the legs it carries enter that one integral through their small-τ endpoint coefficients instead of producing further integrals. At the boundary $\tau \to 0$ the massive legs behave like (3.5) and a massless leg contributes its leading power or a pure exponential, so each one-dimensional integral has the form $\int_{-\infty}^{0} \mathrm{d}\tau\, (-\tau)^A e^{\mathrm{i}K\tau} = (-\mathrm{i})^{A+1}\Gamma(A+1)K^{-A-1}$, which gives a Gamma function; worked examples of the resulting boundary data and of the blow-up charts are given in [39, 57]. The boundary conditions of all master integrals are therefore products of Gamma functions with some overall coefficients, as used in the numerical evaluation of Sec. 4.3.

The construction above presumes that every vertex supplies an energy with which to be ranked in (3.55) and localized in τ_v. A vertex that carries no external leg at all has no such energy: its exponential factor is absent, and the strict-order chart that freezes the thetas is not defined for it. Rather than special-casing this situation, the package adds a private *auxiliary* external-leg energy $k_{0,v}^{\mathrm{aux}}$ to the vertex inside the very same dlog system. The auxiliary symbol is an ordinary coordinate: it enters the energy letters and the component signed energies exactly as a physical $k_{0,v}$ does, so the boundary data are first constructed from the damped limit $K_v^{\mathrm{aux}} \to \infty$ of the augmented system, and the transport then runs to the physical point, where every auxiliary coordinate takes its target value $k_{0,v}^{\mathrm{aux}} = 0$. Here $K_v \equiv \mathrm{i}p_{0,v} = \mathrm{i}\varsigma_v k_{0,v}$ is the *damping energy* of the vertex: on the damping ray $k_{0,v} \to -\mathrm{i}\varsigma_v K_v$ with real positive K_v, and the exponential of (3.15) becomes the decaying $e^{K_v\tau_v}$, which is the quantity ranked in (3.52) and, for a merged component, summed into $K_C = \mathrm{i}p_{0,C}$; the package records this substitution as `dampingDefinition` in the vertex coordinates. All auxiliary coordinates are added to the system and to every requested point at once, so this is a single transport in the enlarged coordinate space rather than a succession of limits. No re-derivation takes place, and the user never supplies or sees this coordinate. Transporting to zero is a limit inside the solution, and it can land on a singular point: if the only nonzero signed energy of some component was auxiliary, its letters vanish at $k_{0,v}^{\mathrm{aux}} = 0$. Such a finite singular point returns either a finite value or an explicit divergence instead of silently producing a number.

3.6 Several propagators sharing one theta

Up to now every contact has been attached to one full propagator. A tree graph can nevertheless contain two or more full propagators whose two vertices lie on the same pair of current components, so that all of them carry the identical theta argument $\Delta = \tau_u - \tau_v$; the package groups exactly such propagators into a *common-theta bundle* $\mathcal{B}$, with $|\mathcal{B}| \geq 2$, and a time-order cycle is the typical source. This case is absent from [57], where each contact is a single propagator. Figure 1 draws the minimal such graph: two vertices, no external legs, and four full propagators that all join the same pair of vertices, the vertical

dots between the inner two propagators marking that any number of further propagators may sit inside the same bundle. The handling is best summarized in four steps, with the details in App. A.

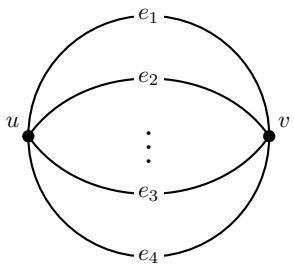


Figure 1. A common-theta bundle of four full propagators. The vertices u and v carry no external legs, and every propagator $e_1, \dots, e_4$ joins this same pair of vertices, hence the same pair of current components at every stage of the reduction. All four propagators therefore carry the identical theta argument $\Delta = \tau_u - \tau_v$ and form one bundle $\mathcal{B}$ with $|\mathcal{B}| = 4$. The two vertices sit at the left and the right end, the outer two propagators make up the halves of one circle, and the vertical dots between the inner two propagators stand for any number of further propagators joining the same pair, so the bundle is not restricted to the four propagators drawn.

Write the bundle as, per propagator e, an ordered pair of products α_e (coefficient of $\theta(\Delta)$) and β_e (coefficient of $\theta(-\Delta)$).

1. *Multiply the thetas out first.* Since inside a bundle all thetas are the same function, $\theta(\Delta)^2 = \theta(\Delta)$ and $\theta(\Delta)\theta(-\Delta) = 0$, the product of $|\mathcal{B}|$ propagators collapses to *two* terms rather than $2^{|\mathcal{B}|}$,

$$\prod_{e\in\mathcal{B}} \Big(\theta(\Delta)\alpha_e + \theta(-\Delta)\beta_e\Big) = \theta(\Delta)\prod_{e\in\mathcal{B}} \alpha_e + \theta(-\Delta)\prod_{e\in\mathcal{B}} \beta_e. \tag{3.58}$$

2. *One theta, one delta.* Differentiating the shared theta yields a single $\delta(\Delta)$ whose coefficient is the antisymmetric difference under $\alpha \leftrightarrow \beta$, $\partial_\Delta\big[\theta(\Delta)\prod \alpha_e + \theta(-\Delta)\prod \beta_e\big] = \delta(\Delta)\big(\prod_e \alpha_e - \prod_e \beta_e\big)$, so all contact information of the bundle is concentrated in one distribution and the two vertices are forced to coincide only once.

3. *Split and change basis.* Telescoping the antisymmetric difference and then collecting in $\mathsf{d}_e = \alpha_e - \beta_e$ and $\mathsf{m}_e = (\alpha_e + \beta_e)/2$ leaves only terms with an odd number of d factors, each with a numerical coefficient,

$$\prod_{e\in\mathcal{B}} \alpha_e - \prod_{e\in\mathcal{B}} \beta_e = \sum_{\substack{\varnothing\neq S\subseteq\mathcal{B}\\ |S| \text{ odd}}} 2^{1-|S|} \prod_{e\in S} \mathsf{d}_e \prod_{e\notin S} \mathsf{m}_e. \tag{3.59}$$

4. *Read off the events.* Every non-empty odd subset S is one simultaneous contact event: the propagators of S are contracted at once, their two vertices merge once, the component count drops by one, and the base power of the merged component

follows (3.36) with $|C|-1$ merge terms. The injection matrix $C^{(v,1)}_{st}$ of (3.45) for such an event is the tensor product of the local selectors of the propagators in S, multiplied by $2^{1-|S|}$ and by the spectator factors m_e.

The collected form (3.59) rather than the term-by-term telescoping is what the program uses, because a term-by-term enumeration would count the same sector transition several times; the argument, the parity proof and the treatment of unselected propagators whose two vertices become coincident are given in App. A.

4 Using the package

4.1 Installation and loading

Before using package we collect the system requirements and the loading commands.

System requirements.

- A Wolfram Language kernel: Mathematica or the free Wolfram Engine (the package is a `.wl` package; no external Wolfram libraries are needed).
- Python 3.10 or newer with `python-flint>=0.6` and `sympy>=1.12` installed, for the numerical backend. MadStree follows a zero-probe policy: the interpreter is selected only by the explicit `PythonExecutable` option or, when that is `Automatic` (the default), by the `MADSTREE_PYTHON` environment variable, falling back to `"python"`.

Install the dependencies in the Python interpreter used for numerical evaluation:

```
python -m pip install "python-flint>=0.6" "sympy>=1.12"
```

Here `python` should be replaced by the selected interpreter if necessary. MadStree includes a copy of FlintNDE under `MadStree/Vendor/FlintNDE`, so a separate installation of the FlintNDE package is not required.

Loading the package. Download or clone the official release repository https://github.com/J7C/dSIBP-Packages, which contains `dSIBP`, `MadStree` and `FlintNDE`. Set `madStreeDirectory` to the `MadStree` directory in the local repository, add it to `$Path`, and load with `Needs`:

```
madStreeDirectory = ".../dSIBP-Packages/MadStree";
AppendTo[$Path, madStreeDirectory];
Needs["MadStree`"];
```

Replace the example path with the absolute path to the local `MadStree` directory. This loads `MadStree.m`, which loads `Kernel/MadStree.wl` with explicit UTF-8. Equivalently, load the kernel file directly:

```
Get[FileNameJoin[{madStreeDirectory, "Kernel", "MadStree.wl"}],
  CharacterEncoding -> "UTF-8"];
```

Loading through `Needs` requires no user encoding option. MadStree uses its bundled FlintNDE backend without requiring `Needs["FlintNDE`"]`. Use a fresh kernel when switching versions, since `Needs` does not reload an already loaded package.

After loading, `MSFlintNDEConfiguration[]` returns an association. The following expression should return `"1.0"`:

```
MSFlintNDEConfiguration[]["version"]
```

The following expression should return `True`:

```
MSFlintNDEConfiguration[]["availableQ"]
```

It confirms that `flintnde/__init__.py` exists under the configured backend directory, which is `Vendor/FlintNDE` relative to `madStreeDirectory` by default.

4.2 Input formats and core functions

Tree diagrams: `MSInitTree`. A tree is given by an association of vertices and propagators. The propagator list sits under the input key `"lines"`, but we call these objects propagators throughout. A tree with two vertices and one massless propagator between them:

```
spec = <|
  "name" -> "masslessFullEdge",
  "vertices" -> {
    <|"id" -> v1, "externalLegEnergy" -> k1,
      "timePower" -> a1, "vertexType" -> "+"|>,
    <|"id" -> v2, "externalLegEnergy" -> k2,
      "timePower" -> a2, "vertexType" -> "+"|>
  },
  "lines" -> {
    <|"type" -> "massless", "endpoints" -> {v1, v2},
      "momentum" -> q, "nu" -> 1/2|>
  }
|>;
context = MSInitTree[spec];
```

Vertex fields:

field	meaning	note
`id`	vertex symbol	required
`externalLegEnergy`	external energy parameter (e.g. `k1`)	optional; default 0
`timePower`	time-power variable a_v (e.g. `a1`)	required
`vertexType`	contour assignment `"+"` or `"-"`	required

Propagator fields:

field	meaning	note
`type`	`"massive"` or `"massless"`	required
`endpoints`	list of one external or two internal vertices	required
`momentum`	momentum variable k_i (e.g. `q`)	required
`nu`	supplied Bessel/Hankel order ν	required for massive propagators; massless default 1/2

A massless propagator joining two internal vertices is classified internally as `masslessFull` and always uses the shared two-state quotient basis of Sec. 3.3; there is no field that selects another massless representation. The initialization option `NuConvention -> "Positive"` (default) or `"Negative"` chooses the prefactor sign of $h(\nu, 0; z) = z^{\pm\nu} H_\nu(z)$ introduced in Sec. 3.1; it is fixed for the whole context and cannot be changed per call.

Propagator numbers are assigned from the input order. The Full, Cross or External class and the contour signs are derived from the `vertexType` of the two vertices the propagator joins; they are not supplied as a type value or as separate sign fields. A `"+"` vertex has phase $e^{-\mathrm{i}E\tau}$ and a `"-"` vertex has phase $e^{+\mathrm{i}E\tau}$, where E is `externalLegEnergy`. An omitted or zero external energy is represented internally by an auxiliary variable whose target value is zero. Full propagators that join the same pair of vertices are grouped into one common-theta bundle automatically; the optional top-level field `"thetaBundles"` pins such a group explicitly as a list of propagator ids, and any group whose members do not join the same pair of vertices is rejected.

Loop diagrams without loop integral: `MSInitTimeGraph`. For a pure time-order graph the vertex and propagator fields are the same, but the graph may contain cycles. A triangle cycle reads

```
cycleContext = MSInitTimeGraph[<|
  "vertices" -> {
    <|"id" -> t1, "externalLegEnergy" -> k1,
      "timePower" -> a1, "vertexType" -> "+"|>,
    <|"id" -> t2, "externalLegEnergy" -> k2,
      "timePower" -> a2, "vertexType" -> "+"|>,
    <|"id" -> t3, "externalLegEnergy" -> k3,
      "timePower" -> a3, "vertexType" -> "+"|>
  },
  "lines" -> {
    <|"type" -> "massless", "endpoints" -> {t1, t2},
      "momentum" -> q12, "nu" -> 1/2|>,
    <|"type" -> "massless", "endpoints" -> {t2, t3},
      "momentum" -> q23, "nu" -> 1/2|>,
    <|"type" -> "massless", "endpoints" -> {t3, t1},
      "momentum" -> q31, "nu" -> 1/2|>
```

```
  }
|>];
```

Here every propagator joins two internal vertices, so all three are classified internally as `masslessFull`. Once one of them is contracted, the remaining two connect the same pair of components and therefore form the common-theta bundle of Sec. 3.6.

Single-vertex integral families: `MSInitVertexFamily`. No fake topology is needed for a single vertex. The compact input follows the reference `ki/nui` convention,

```
vertexContext = MSInitVertexFamily[<|
  "ki" -> {k0, k1, k2},
  "nui" -> {a0, nu1, nu2},
  "hankelBranches" -> {1, 2}
|>, NuConvention -> "Positive"];
```

where `First[ki]` is the pure vertex phase energy and `First[nui]` the base power of $(-\tau)$; the remaining entries give, position by position, the h-block momenta and their Hankel orders ν, which are used verbatim as in Sec. 3.1. The explicit alternative `externalLegEnergy/timePower/hBlocks/exponentialBlocks` is also accepted, where each h-block is an association with the keys `momentum`, `nu` and `hankelBranch`. Unlike tree and time-graph input, `MSInitVertexFamily` rejects any field outside its own key set.

Package conventions. The package implements the h-states of (3.8) with $z = -k\tau$ and the single function name h, using

$$h(\nu, 0; z) = z^{\pm\nu} H_\nu(z), \tag{4.1}$$

where the plus sign is the default (`NuConvention->"Positive"`) and the minus sign gives the convention of [57]; the two bases are related by $\nu \to -\nu$ (Sec. 3.1). The master integrals are the $I_{\tilde{\mathbf{a}}}$ of (3.11); for multi-vertex correlators they are the products and subsector integrals of Sec. 3.3. Each master integral is represented by `MSIntegral[sectorKey, timeShifts, stateBits]`: `sectorKey` is a binary string over the root propagators in the input order, with `1` for a propagator that is still active and `0` for a propagator contracted by a contact; `timeShifts` assigns one integer shift of the base time power to every vertex component, where a component is a maximal set of vertices merged by the contractions; `stateBits` gives the state of every two-state slot of the sector in the slot order. The master integrals are the entries with vanishing shifts, and their connection is the d log-form (3.31) of Sec. 3.3.

Let us see some examples for `MSIntegral[sectorKey, timeShifts, stateBits]`. For the three-vertex chain v_1–v_2–v_3 with internal propagators l_{12}, l_{23}, vertex energies k_1, k_2, k_3 and time powers a_1, a_2, a_3, the contact-reachable sectors are the four strings `"11"`, `"01"`, `"10"`, `"00"`,

(i) both propagators massive: l_{12} with ν_{12}, l_{23} with ν_{23}.

sectorKey	contracted	components	master integrals
"11"	—	$\{v_1\},\{v_2\},\{v_3\}$	MSIntegral["11",{0,0,0},{a,b,c,d}] (16)
"01"	l_{12}	$\{v_1,v_2\},\{v_3\}$	MSIntegral["01",{0,0},{c,d}] (4)
"10"	l_{23}	$\{v_1\},\{v_2,v_3\}$	MSIntegral["10",{0,0},{a,b}] (4)
"00"	l_{12},l_{23}	$\{v_1,v_2,v_3\}$	MSIntegral["00",{0},{}] (1)

(ii) l_{12} massive with ν_{12}, l_{23} massless.

sectorKey	contracted	components	master integrals
"11"	—	$\{v_1\},\{v_2\},\{v_3\}$	MSIntegral["11",{0,0,0},{a,b,r}] (8)
"01"	l_{12}	$\{v_1,v_2\},\{v_3\}$	MSIntegral["01",{0,0},{r}] (2)
"10"	l_{23}	$\{v_1\},\{v_2,v_3\}$	MSIntegral["10",{0,0},{a,b}] (4)
"00"	l_{12},l_{23}	$\{v_1,v_2,v_3\}$	MSIntegral["00",{0},{}] (1)

(iii) both propagators massless.

sectorKey	contracted	components	master integrals
"11"	—	$\{v_1\},\{v_2\},\{v_3\}$	MSIntegral["11",{0,0,0},{r,s}] (4)
"01"	l_{12}	$\{v_1,v_2\},\{v_3\}$	MSIntegral["01",{0,0},{r}] (2)
"10"	l_{23}	$\{v_1\},\{v_2,v_3\}$	MSIntegral["10",{0,0},{r}] (2)
"00"	l_{12},l_{23}	$\{v_1,v_2,v_3\}$	MSIntegral["00",{0},{}] (1)

The vertex `vertexType` field (`"+"` or `"-"`) determines the sign of the pure vertex exponential $e^{\mathrm{i}k_0\tau}$ of (3.11), and the pair of vertex types determines the SK class of each propagator, from which the default Hankel branches of (3.8) follow; a propagator may override them with its own `hankelBranches` field. The package h-states are those of (3.8), i.e. $h(\nu,1;z)=\partial_z h(\nu,0;z)$, with the alternative prefactor $z^{+\nu}$ obtained by $\nu\to-\nu$; `MSHTohMatrix`, `MShToHMatrix` and `MSConvertBasis` implement the corresponding basis change and its Kronecker products, with the massless shared two-state of Sec. 3.2 acting trivially on the reduced states.

Once the context is initialized, the following functions expose the formula products of Sec. 3.

Core functions. Most functions take the initialized `context`; `MSExportEvaluationData` takes a completed evaluation result instead.

function	purpose
`MSSectors[context]`	contact-reachable sectors in deterministic order (keys, pinched propagators, master counts)
`MSMasterIntegrals[context]`	ordered master records; each `"integral"` field is an `MSIntegral[sectorKey, timeShifts, stateBits]`
`MSFormulaMatrices[context, key]`	per-component M_1, M_0 and energy letters $\kappa_{\boldsymbol{a}}$; sector diagonalizer `U` and `UInverse`
`MSDLogDE[context]`	block-triangular d log-form potential Ω, letters, certification status
`MSReduce[expr, context]`	iterative IBP reduction to the master basis; scalar, list or nested table input
`MSContactMaps[context, key]`	parent-to-sub-sector contact maps
`MSWriteFormulaArtifacts[context]`	write formula products beside the caller
`MSFormulaData[context]`	aggregate all-sector masters, recurrence metadata and the d log-form DE
`MSEvaluatePath[context, pointSequence, opts]`	single-stage evaluation: boundary initialization and segment transport in one backend process; optional FlintNDE path planning
`MSReconstructEpSeries[context, ep, pointSequence, opts]`	Laurent series in the common analytic regulator: certified lowest power, automatic sample points, incremental fitting
`MSExportEvaluationData[evaluation, opts]`	export the saved ordinary-point records of an evaluation to CSV/JSON

`MSSectors[context]` returns the sector list. Each sector key is the fixed-length bit string over the root propagators in input order (`"0"` pinched, `"1"` unpinched); the sub-sector structure is discussed in Sec. 3.3. Keys are strings, not integers; leading zeros are part of the identity.

`MSMasterIntegrals[context]` returns the ordered master records. Their `"integral"` fields contain the `MSIntegral[sectorKey, timeShifts, stateBits]` objects of Sec. 4.2, i.e. the master integrals $I_{\tilde{\boldsymbol{a}}}$ of (3.11). Use `Lookup[MSMasterIntegrals[context], "integral"]` to obtain this ordered basis. Each record also contains `"bareIntegral"`, `"normalization"` and `"definition"`. Their order is the unique authority for all matrices, boundary vectors and numerical results.

`MSFormulaMatrices[context, key]` returns the component records under `"components"`, with fields `"M1"`, `"M0"` and `"energyLetters"` for the matrices and letters of Sec. 3.2. The sector diagonalizer of M_0 and its inverse are returned as `"U"` and `"UInverse"`; M_1 is already diagonal in the state basis. Here `key` is a

sector-key string; omitting it, or passing `All`, returns an association for all sectors. `MSContactMaps` accepts the same sector selection.

`MSDLogDE[context]` returns `omegaPotential`, the matrix Ω of Sec. 3.3 whose derivative along a path is the DE matrix, together with the letters (the arguments of the logarithms of (3.32)), `letterMatrices`, `dlogResidual`, `constantResidueQ` and the certification `dlogStatus`. Only `"certifiedByFormulaChecks"` is a closed d log-form and may be consumed numerically.

`MSReduce[expr, context, MasterBasis -> Automatic]` reduces finite linear combinations of legal `MSIntegrals`, or lists and nested tables of such expressions, by the lowering/raising recurrences $A_{\pm}$ of Sec. 3.2 along the sub-sector structure of Sec. 3.3. Results include `result`, `masterBasis`, `coefficientVector`, `masterRules`, `nonMasterResidual`, `remainingShiftedIntegrals` and `singularLayers`; a full reduction requires a zero residual and an empty shifted list for every scalar expression. For list input, these fields preserve the input shape; `elementResults`, `failurePositions` and `incompletePositions` provide the element-level reports.

`MSFormulaData[context]` returns all-sector masters, the per-sector $\mathrm{M}_1/\mathrm{M}_0/\mathrm{T}_n$ recurrence metadata of Sec. 3.2 and the complete block-triangular d log-form DE in the master order of `MSDLogDE`; `TimePowerRules` may substitute the vertex time powers a_i in the recurrence metadata, but the returned `"dlogDE"` remains analytic and unsubstituted.

`MSEvaluatePath[context, pointSequence, opts]` is the single-stage numerical entry for evaluating the master integrals at the user points. `pointSequence` is a coordinate table: its first row gives the ordered coordinate symbols and subsequent rows give equal-width coordinate values. Fixed parameters are supplied once through `ParameterRules`. Boundary generation, transport and result records are described in Sec. 4.3; the principal options and point tags are summarized in Sec. 4.5.

`MSReconstructEpSeries[context, ep, pointSequence, opts]` reconstructs the Laurent series of the master integrals in a common analytic regulator `ep`, with `MaximumEpPower` the highest returned power (default 0, i.e. the pole terms and the finite part). It uses the same `pointSequence` and `ParameterRules` format as `MSEvaluatePath`. Only `ParameterRules`, not the coordinates, may depend on `ep`. By default, no `ep` values need to be supplied; `EpSamplePoints` and `EpValidationPoints` may instead specify production candidates and disjoint validation points. The certificate conditions, sampling and fitting procedure are described in Sec. 4.4; the corresponding options are summarized in Sec. 4.5.

`MSExportEvaluationData[evaluation, opts]` exports the saved ordinary-point records of one `MSEvaluatePath` result, including the coordinates, the real/imaginary parts of every master, `status`, `relativeDifferenceInf` and `targetRelativeErrorMet`, to CSV and JSON for downstream plotting tools; transient records are excluded. The two precision fields are the corresponding segment reports, not separate error

estimates computed at every point. `ExportFormats` selects CSV, JSON or both, and `SignificantDigits` defaults to 16; see Sec. 4.2.

Output organization.

- Transient request JSON and backend logs: `results_temp/nde/` beside the caller by default. Request and log files are deleted after a successful run and retained for backend failures; structured singular-path or leading-order refusals also remove these files. Successful JSON results remain in `results_temp/cache/` for reuse. `MSRuntimeDirectory` may specify a different runtime directory.

- Saved point records: `MSEvaluatePath` returns the records under `"pointResults"`. An ordinary untagged coordinate row has `"status" -> "saved"`; `"saved"` is a status value, not a top-level key. A row {{`values...`}, `"tmp"`} gives a transient waypoint, represented by `"status" -> "transient"` and excluded from export. Singular-point classifications are returned under `"singularityClassifications"` by the same backend evaluation run.

- Formula artifacts: `MSWriteFormulaArtifacts[context]` writes `masters.wl`, `recurrence_metadata.wl`, `dlog_de.wl` and `manifest.wl` to `results/madstree_formula/run-UUID/` by default. `MSEvaluatePath` and `MSReconstructEpSeries` automatically write or reuse these analytic files before numerical transport and return their paths under `"formulaArtifacts"`.

- Evaluation export: `MSExportEvaluationData[result, MSOutputDirectory -> ...]` writes `evaluation_data.csv` and `evaluation_data.json` under the caller-chosen output directory (default `results/madstree_evaluation/run-UUID/`), for downstream plotting tools.

- Cache identity: a successful-result cache key includes the request and the SHA-256 identities of the backend adapter, the configured backend's `pyproject.toml` and sorted `flintnde/*.py`, so in-place source fixes invalidate stale cache entries automatically.

The package source directory never receives run artifacts.

4.3 Numerical evaluation

With an initialized context, numerical evaluation proceeds in two steps: boundary generation and transport. The workflow is single-stage: `MSEvaluatePath` partitions the user point sequence into maximal consecutive complex-affine one-variable segments, pulls the d log-form connection back once per segment, and sends all segments, the exact boundary conditions and the user options to FlintNDE in one UTF-8 JSON request. With the default `FlintNDEPathPlanning -> True`, FlintNDE plans the transport nodes inside each segment and evaluates user points by fast multipoint evaluation; with `False` it uses every supplied point as a node in input order, without inserting, removing, or replanning

points. The backend inside the version directory is a synchronized copy of the standalone `FlintNDE` package released in the same repository, whose mechanisms are described below. `MSBoundaryData` and `MSEvaluatePath` default to 200 decimal working digits; an explicit `WorkingPrecision` overrides the default.

Boundary generation.

```
targetRules = {k1 -> 9 I, k2 -> 3 I, q -> 1, a1 -> 1, a2 -> 1};

certificate = MSBoundaryChartCertificate[context, targetRules,
  RankOrder -> All];

boundary = MSBoundaryData[context, targetRules,
  BoundaryScale -> 4, WorkingPrecision -> 200];
```

`MSBoundaryChartCertificate` verifies the nested blow-up chart of Sec. 3.5 (normal crossing, theta fixing, quotient checks) for the chosen strict energy rank; `RankOrder -> All` checks every strict chart. `MSBoundaryData` consumes the certificate and returns the leading Frobenius data at the $K_v \to \infty$ boundary: for every master whose leading weight does not vanish, one branch record carrying the Frobenius exponent, the overall coefficient and the normalized leading vector, together with the anchor rules, the rank order and the certificate itself. If the chart is not certified, it returns `BoundaryChartNotCertified` and does not proceed to numerics. The energies in the example are positive imaginary so that the vertex factors $e^{-\mathrm{i}E\tau}$ damp at $\tau \to -\infty$; the rank order of the boundary is derived from the supplied target point, largest damping energy first, unless `RankOrder` fixes it. The example passes `WorkingPrecision -> 200` explicitly; 200 is also the default, and any explicit positive value overrides it.

Transport.

```
pointSequence = {{k1, k2}, {9 I, 3 I}};
parameterRules = {q -> 1, a1 -> 1, a2 -> 1};

result = MSEvaluatePath[context, pointSequence,
  ParameterRules -> parameterRules,
  BoundaryScale -> 4, RankOrder -> {v1, v2},
  WorkingPrecision -> 200, TransportOrder -> 80,
  ReferenceTransportOrder -> 104,
  TargetRelativeError -> "1e-20",
  MessageLanguage -> "EN"];

result["pointResults"]
```

The second argument is the *point sequence*: its first row lists the movable coordinate symbols, and every further row supplies one numerical value for each of them, in the same

order. The remaining symbols of the target point, here `q`, `a1` and `a2`, are fixed once for the whole run through `ParameterRules`. For several targets, append one row per point,

```
pointSequence = {{k1, k2}, {8 I, 2 I}, {7 I, 4 I}, {6 I, 5 I}};
```

and MadStree groups the points into maximal consecutive complex-affine segments and pulls each group back once, while FlintNDE evaluates every point in one process, including dense points inside a node's convergence disk, without changing the path. For a single-vertex integral family, run the same single-stage entry on the vertex context. The principal transport options and the point tags are summarized in Sec. 4.5. If every d log-form letter is constant on a segment, every pulled-back derivative vanishes; the segment is then a zero connection with no finite poles and is transported normally.

Reading the result. `result` is an association whose `"pointResults"` entry is the list of point records. Every saved ordinary point carries `coordinate`, `value` (the master vector in master order), `status` (equal to `"saved"` for ordinary points) and `userIndex`; transient `"tmp"` points participate only in the path. The current version does not provide leading-order point labels: every tag other than `"tmp"` is rejected. With the default `SingularityMode -> "Automatic"` a segment that meets a supported singularity is crossed by an explicit jump; `"Avoid"` refuses such a segment instead, and with `FlintNDEPathPlanning -> False` the backend is forced into avoid mode, so a user chain that crosses a pole or leaves a local convergence disk fails explicitly. The exported per-point `relativeDifferenceInf` and `targetRelativeErrorMet` are obtained from the embedded truncation certification of Sec. 4.3, not from a second independent transport chain. A route-robust access to a saved value is

```
savedPoints = Select[result["pointResults"], #["status"] === "saved" &];
firstPoint = First[savedPoints];
firstPoint["value"]
```

The numerical backend. FlintNDE is a general solver for first-order matrix differential equations with exact $Q(\mathrm{i})(s)$-coefficient systems; MadStree embeds a synchronized copy and only ever feeds it single-variable pullbacks of the d log-form connection. Three mechanisms are responsible for the current performance:

- *Pole–residue transport.* An ordinary segment is serialized as its pole–residue representation $A(x) = P(x) + \sum_j R_j/(x - p_j)$, with P of arbitrary degree and the residues R_j of coincident letters merged. The backend generates the solution Taylor coefficients by a pole-state recurrence, avoiding the Cauchy sampling and discrete-Fourier coefficient reconstruction of a generic analytic-matrix backend.

- *Embedded truncation certification.* Only one transport chain is run (`certificationMode -> "embedded"`); the primary result is re-evaluated from the coefficient prefix by Horner, and the per-segment truncation difference is returned as a local error estimate, so the reference-chain overhead is removed while the `relativeDifferenceInf` and `targetRelativeErrorMet` fields remain available for the user.

- *Fast multipoint evaluation.* With `FlintNDEPathPlanning -> True`, FlintNDE plans nodes inside each segment; user points covered by one expansion node form an evaluation bucket and are evaluated from the cached vector series. Buckets with at least eight points use the subproduct/remainder tree of fast multipoint evaluation, and small buckets use the iterative algorithm to avoid tree overhead. User points in the same complex-affine group $x(s) = x_0 + s\,v$ share node coefficients whenever they lie in the same convergence disk; different segments never share local coefficients.

Singularity handling is explicit: the default `SingularityMode -> "Automatic"` maps to the backend jump mode, so a segment that meets a supported singularity is crossed by an explicit local-basis jump instead of being refused or detoured; `"Avoid"` refuses such a segment, and `"SingularityJump"` requests a jump unconditionally. Both jump modes are available only when `FlintNDEPathPlanning -> True`; with planning disabled the backend is forced into avoid mode and the user must supply a valid ordinary-point node chain. The branch reached by a jump is fixed by the local basis and by the winding of the path, and the user must check that it is the intended one. Exact Lee–Moser balances, strictly decoupled exponential times, and power-log recurrences are handled, while resonances whose local bases cannot be constructed, and any ramification or general Stokes requirement, fail closed. The internal working bits are $\lceil \mathrm{WP}\log_2 10\rceil + 32$ for `WorkingPrecision` = WP digits (the default WP is 200), and all path geometry (segment projections, match ratios, rotation factors, winding/monodromy) is kept in Arb/Acb precision. If the backend realification writes $k = -\mathrm{i}\kappa$, then $-k^2 = +\kappa^2$; a program token named `ik` then denotes the real variable κ, and `ik^2` is its square, not the imaginary unit times k^2.

4.4 Laurent-series reconstruction

When all vertex time powers share a common analytic regulator ε, e.g. $a_v = 1 + \varepsilon$, the master integrals develop a Laurent series

$$I(\varepsilon) = \sum_{p=n_{\min}}^{m} c_p\,\varepsilon^p + O(\varepsilon^{m+1}), \tag{4.2}$$

and `MSReconstructEpSeries[context, ep, pointTemplate, MaximumEpPower -> m]` reconstructs the coefficients $c_{n_{\min}}, \ldots, c_m$. The user supplies no ε values; the lowest power is not a numerical fit parameter.

This route follows the one used by `AMFlow` [54, 55]. The transport is numerical, while the regulator dependence of the result has to stay analytic, and carrying that analytic dependence through every matrix multiplication of a long path would dominate the cost. Each sample is therefore solved purely numerically at a fixed value of ε, and the coefficients of (4.2) are recovered afterwards from the final numbers of the whole set of runs, which is what keeps a multi-order expansion affordable.

Certification of the lowest power. Before any numerical solve at nonzero ε, MadStree constructs a support certificate from the symbolic boundary conditions and the d log-form DE of the production context. It requires:

1. regulator-independent path coordinates and no negative Laurent power at $\varepsilon = 0$ in any d log-form matrix, pulled-back connection, or residue;

2. finite-Laurent boundary branches, where branches with the same limiting Frobenius exponent and log degree are first combined with their physical weights and leading vectors, and only then are component valuations extracted;

3. at least one provably nonzero coefficient at the lowest power of the combined boundary; a noninteger power, non-Laurent structure, undecidable coefficient, or support entirely above the requested user range fails closed.

For $n_{\min} < 0$ the certificate describes a finite meromorphic Laurent boundary, not an analytic boundary at $\varepsilon = 0$; analyticity is recorded only when $n_{\min} \geq 0$. Along the current finite path an ε-analytic invertible fundamental matrix preserves the valuation of nonzero formal boundary conditions, so the certified $n_{\min}$ is passed directly to the sampling plan. A rank drop of an individual Frobenius recurrence operator at $\varepsilon = 0$ is retained as a resonance diagnostic only; it is not by itself interpreted as a $1/\varepsilon$ pole of the already combined physical boundary.

Sampling, fitting and validation. Given the highest requested power m and the certified lowest power $n_{\min}$, the plan sets

$$P = \max(0, -n_{\min}), \qquad \Delta = m - n_{\min}, \qquad N_{\text{fit}} = \max\left(\left\lceil \tfrac{5}{2}\Delta + P \right\rceil, \Delta + 1 + f\right), \quad (4.3)$$

where $f =$ `EpFitExtraOrder`, default 2, is the number of powers the fit keeps above the user maximum: the second branch of (4.3) would be $\Delta + 1$ if only $c_{n_{\min}}, \ldots, c_m$ were fitted, and the extra f is what makes the interpolation reach past c_m in every run. `SampleCount` may raise N_{fit} further, never below $\Delta + 1$. The plan then chooses the ε scale and precisions

$$\alpha_\varepsilon = \frac{P}{4} + \frac{g}{\Delta + 1}, \qquad \varepsilon_0 = 10^{-\alpha_\varepsilon}, \qquad p_0 = \max(\lceil (N_{\text{fit}} + P)\alpha_\varepsilon \rceil, 30), \quad (4.4)$$

where $g =$ `EpGoalDigits` is the requested decimal accuracy. The working precision is $\max(200, \lceil 2(1 + e)p_0 \rceil)$ with e the extra-precision fraction; the primary transport order is $4p_0$ and the reference chain adds $\lceil \max(50, p_0/5) \rceil$ orders. All N_{fit} production points enter one square Acb Vandermonde system interpolating through $\varepsilon^{n_{\min}+N_{\text{fit}}-1}$; only $c_{n_{\min}}, \ldots, c_m$ are returned, and no least squares or pseudoinverse is used. Independent validation points are placed at a fixed fraction of the production scale, never enter the fit, and the truncated series must pass the relative tolerance 10^{-g}; otherwise the run fails closed.

Incremental expansion and parallelism. The initial fit includes at least two powers above the user maximum (`EpFitExtraOrder -> 2`). If validation fails, each subsequent round adds `EpFitOrderIncrement -> 2` powers, solves only the new production points, and reuses every previous production and validation value, for at most `EpFitMaximumRounds -> 3` rounds; the tolerance is not relaxed and the caches are not discarded. Independent ε tasks run in an outer process pool bounded by `ParallelTaskCount`

-> 12, whose effective concurrency is the smaller of the limit and the number of tasks; this is separate from the in-process ctx.threads of python-flint.

4.5 Options, validation and common pitfalls

The principal options of the numerical workflow are summarized below.

option	meaning
FlintNDEPathPlanning	True (default): FlintNDE plans nodes and uses fast multipoint evaluation; False: user points as nodes in input order
SingularityMode	"Automatic" (default), "Avoid" or "SingularityJump"
MessageLanguage	"EN" (default) or "CN", case-sensitive
BoundaryScale	scale setting the damping energies at the finite boundary anchor (> 1; default 8)
BoundarySeriesOrder	boundary-order metadata and regulator recurrence diagnostics; not the numerical Frobenius start order in the current generic route
RankOrder	vertex order by damping energy K_v (default Automatic: from the first target point)
WorkingPrecision	integer working precision in decimal digits, at least 20; default 200
TransportOrder / ReferenceTransportOrder	primary and reference local-series orders, including the numerical Frobenius start; defaults 48 and 64, with the reference order strictly larger
TargetRelativeError	target relative error of the primary/reference check; default "1e-25"
MaximumEpPower	highest returned regulator power of MSReconstructEpSeries (default 0)
EpGoalDigits	requested decimal accuracy of the reconstructed regulator coefficients (default 20)
EpFitExtraOrder / EpFitOrderIncrement / EpFitMaximumRounds	internal buffer powers, per-round increment and maximum refit rounds of the regulator fit (defaults 2, 2, 3)
ParallelTaskCount	maximum concurrent ep backend processes in MSReconstructEpSeries (default 12)
PythonExecutable	Python command for the FlintNDE adapter (default Automatic)
MSRuntimeDirectory	temporary-file root (default: results_temp in the calling script or notebook directory)

The transport orders and TargetRelativeError above apply to MSEvaluatePath. For MSReconstructEpSeries, these three options are not accepted; the numerical settings are

planned from `EpGoalDigits`. Its `BoundarySeriesOrder` defaults to `Automatic`, whereas `MSEvaluatePath` uses 24.

Saved and transient points are not a transport option: they are tags attached to coordinate rows below the symbol header (bare value lists and {{`values...`},`"tmp"`}). With `SingularityMode -> "Avoid"`, interior singularity crossings are refused and reported with the offending path pairs. The default `"Automatic"` mode allows supported singularity jumps when `FlintNDEPathPlanning -> True`; an explicit `"SingularityJump"` also requires this setting. When planning is disabled, `"Automatic"` uses `"Avoid"`. The user must check the selected multivalued branch. An explicitly requested terminal singular point is handled by local classification and evaluation, rather than by ordinary Taylor transport through the singularity.

Limitations and fail-closed guarantees:

- No loop-momentum integration, no sampled IBP, no Kira or external reduction.
- `dlogStatus` must be `"certifiedByFormulaChecks"`; otherwise the DE is not consumed.
- Uncertified boundary charts and singular points beyond the backend's certified capabilities all fail closed; resonances whose local bases cannot be constructed return structured refusals, and ordinary Taylor transport is never used across singular points.
- An incomplete `MSReduce` call may return an association with `"status" -> "partiallyReduced"` or `"status" -> "failed"`, rather than a top-level `Failure`. Inspect `"remainingShiftedIntegrals"` and `"nonMasterResidual"`.
- For `MSEvaluatePath`, check `result["flintNDE"]["targetRelativeErrorMet"]`, not only `result["status"]`. For `MSReconstructEpSeries`, check `"precisionTargetMet"`; coefficients may be returned with `"status" -> "computed_with_warning"` if the target is not met.
- The production boundary never uses defining integrals; finite-point defining integrals exist only inside independent validation programs.

Finally, we collect the most common mistakes.

Common pitfalls.

1. **Convention**: `NuConvention` is chosen at initialization only (`"Positive"` by default). To compare with the formulas of [39], initialize with `NuConvention -> "Negative"`, which selects the minus sign in (4.1).
2. **Sector keys**: fixed-length strings; leading zeros and the string type are part of the identity (Sec. 3.3).
3. **Single-stage workflow**: call `MSEvaluatePath` directly. `MadStree` in this version does not provide `MSGeneratePath`, `MSEvaluatePlannedPath`, public plan objects, old JSON schemas or any migration wrapper; the default `WorkingPrecision` is 200 and an explicit value overrides it.

4. **Singularity handling**: the default is `SingularityMode -> "Automatic"`, where `"Automatic"` is a string, not the symbol `Automatic`. Use `"Avoid"` to refuse interior crossings. Supported jumps require `FlintNDEPathPlanning -> True`, and the user must check the multivalued branch.

5. **Point tags**: after the coordinate-symbol header, only bare value lists and {{`values...`},`"tmp"`} are accepted; every other tag, including `"lo"`, is rejected.

6. **Backend environment**: MadStree uses zero probing; the interpreter is given by the `PythonExecutable` option or, when `Automatic`, by the `MADSTREE_PYTHON` environment variable (falling back to `"python"`), and it requires Python 3.10 or later with `python-flint>=0.6` and `sympy>=1.12`, and must be able to import the embedded `flintnde`. Backend failures report captured output through messages such as `backendLaunchFailed` and `backendRunFailed`; retained runtime logs contain the details. Successful calls and handled singular-path refusals remove their temporary input and log files.

7. **Message language**: `MessageLanguage` accepts only `"EN"` and `"CN"` (case-sensitive), with `"EN"` the default.

8. **Encoding**: loading through `Needs` uses explicit UTF-8 internally and needs no user encoding option; if loading the kernel file directly fails with syntax errors, set `$CharacterEncoding = "UTF-8"` before loading.

5 Examples

This section collects what ships with the release and what has been computed with it. Sec. 5.1 lists the seven examples distributed with the package, which exercise recurrence reduction, single-vertex families, time-order charts, massless and massive trees, zero external-leg energy, and the regulator reconstruction of Sec. 4.4. Sec. 5.2 presents two numerical applications: the tree with three vertices and two massive exchanges, whose transported results are compared with the multivariable hypergeometric representation of [25], and the bubble time kernel evaluated at fixed internal momenta. The comparisons standing behind the independent validation tasks recorded in Sec. 2, namely the single-vertex families and the massive two-vertex correlator against [39] and the IBP relations and differential equations against the `dSIBP` package [59], are not reproduced here.

5.1 Package examples

The distribution ships seven examples that exercise the complete workflow:

- `01_massless_full_edge.wl`: the massless shared two-state simplification of Sec. 3.2, masters, recurrence, d log-form DE and the automatic boundary/numerical entry; it also demonstrates the single-stage multipoint workflow, with CSV/JSON export;

- `02_single_vertex_family.wl`: single-vertex integral family input, local tensor inverses (Sec. 3.2), and finite-combination reduction;

- `03_time_only_cycle_chart.wl`: time-order cycle input, common-theta contacts (Sec. 3.6 and App. A), and all strict-rank chart certificates; contacts are discussed in Sec. 3.3;
- `04_three_vertex_tree.wl`: a three-vertex, two-propagator massless tree (+++ vertex structure), following the same flow as example 01, with a single-stage multipoint evaluation and CSV/JSON export;
- `05_massive_three_vertex_tree.wl`: a three-vertex, two-propagator massive tree (+ + + vertex structure) with non-half-integer `nu12=3/4`, `nu23=1/3`, followed by a single-stage multipoint evaluation and CSV/JSON export;
- `06_massless_three_vertex_ep_regularization.wl`: a three-vertex massless tree with the common analytic regulator $a_1 = a_2 = a_3 = 1 + \varepsilon$; the user requests only the highest regulator power (`MaximumEpPower -> 0`), and the program certifies the lowest power from the symbolic boundary and DE before choosing production/validation points and extracting the pole and finite part;
- `07_zero_external_leg_energy.wl`: zero external-leg energy, represented through an auxiliary energy with zero target value.

In the massless three-vertex example (Example 06) with $a_1 = a_2 = a_3 = 1 + \varepsilon$, the physical boundary weights contain $\Gamma(2+\varepsilon)^3$, $\Gamma(2+\varepsilon)\Gamma(4+2\varepsilon)$ and $\Gamma(6+3\varepsilon)$, all regular at $\varepsilon = 0$, and the nine-dimensional DE has no negative ε power; the certificate therefore gives $n_{\min} = 0$ before any numerical solve, and no artificial $1/\varepsilon$ term is introduced.

Each example ends with a failure gate: it prints `Example PASSED` and exits 0 only when every checked result is a non-`Failure` object, and exits 1 with the reason otherwise. Run them in a Mathematica front end section by section, or with

```
wolframscript -file .../Examples/01_massless_full_edge.wl
```

5.2 Application examples

We illustrate the MadStree v1.0 workflow with two examples, using FlintNDE v1.0 for numerical transport. All timings measure elapsed time on a personal computer.

Tree with three vertices and two massive scalars exchanging The first example is the chain with three vertices in figure 2.

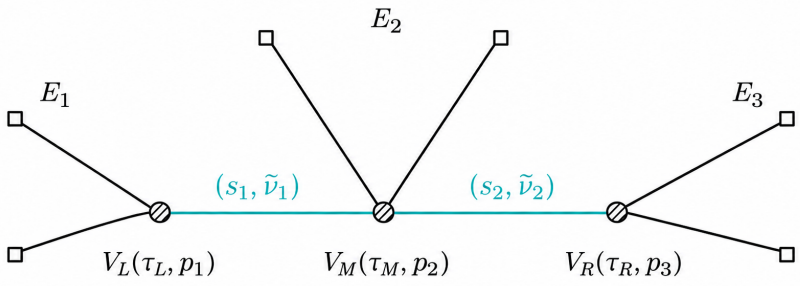


Figure 2. Tree with three vertices and two massive scalar propagators. The external energy sums are E_1, E_2, E_3. The internal propagators carry momentum magnitudes s_1, s_2 and mass labels $\widetilde{\nu}_1, \widetilde{\nu}_2$.

The vertices L, M, R have conformal times τ_L, τ_M, τ_R. The two internal propagators connect L to M and M to R, respectively. Each E_i is the sum of the external momentum magnitudes at vertex i. The internal momentum magnitudes are s_1, s_2, and the Hankel orders are ν_1, ν_2. For the masses in the principal series used here, $\nu_i = \mathrm{i}\widetilde{\nu}_i$, with real $\widetilde{\nu}_i$. Each vertex has a contour sign $\sigma_v = \pm 1$.

We first define the states at the two vertices of a propagator that label the master integrals. This application is evaluated in the `NuConvention -> "Negative"` option, i.e. with $s_\nu = -1$ in the unified definition of Sec. 3.1, because the mode functions that this application starts from follow [39], where they are written in that option:

$$\begin{aligned} h^{(r)}(\nu, 0; z) &= z^{-\nu} H^{(r)}_{\nu}(z), \\ h^{(r)}(\nu, 1; z) &= \partial_z h^{(r)}(\nu, 0; z) = -z^{-\nu} H^{(r)}_{\nu+1}(z), \qquad r = 1, 2. \end{aligned} \tag{5.1}$$

Nothing here is a third convention: the package default is $s_\nu = +1$, with $h^{(r)}(\nu, 0; z) = z^{+\nu} H^{(r)}_{\nu}(z)$ and $h^{(r)}(\nu, 1; z) = z^{+\nu} H^{(r)}_{\nu-1}(z)$, and the two readings are related by the single simultaneous replacement $\nu \to -\nu$ of Sec. 3.1, which must also be applied to the effective time powers below. Here r selects the Hankel function and the binary index selects the undifferentiated or differentiated state. We use the same Hankel order at both vertices of each propagator; constant factors from the physical mode normalisation are restored when forming the correlator. To display the time ordering explicitly, define

$$\begin{aligned} W_{>,\nu;ab}(s; \tau, \tau') &= h^{(1)}(\nu, a; -s\tau) h^{(2)}(\nu, b; -s\tau'), \\ W_{<,\nu;ab}(s; \tau, \tau') &= h^{(2)}(\nu, a; -s\tau) h^{(1)}(\nu, b; -s\tau'), \qquad a, b \in \{0, 1\}. \end{aligned} \tag{5.2}$$

The corresponding propagator kernels, with mode prefactors removed, are

$$\begin{aligned} \mathcal{G}_{++,\nu;ab} &= \theta(\tau - \tau') W_{>,\nu;ab} + \theta(\tau' - \tau) W_{<,\nu;ab}, \\ \mathcal{G}_{--,\nu;ab} &= \theta(\tau - \tau') W_{<,\nu;ab} + \theta(\tau' - \tau) W_{>,\nu;ab}, \\ \mathcal{G}_{+-,\nu;ab} &= W_{<,\nu;ab}, \qquad \mathcal{G}_{-+,\nu;ab} = W_{>,\nu;ab}. \end{aligned} \tag{5.3}$$

All kernels in this equation have arguments $(s;\tau,\tau')$. The binary indices act only on the Hankel factors, not on the step functions.

For $\boldsymbol{\sigma} = (\sigma_L, \sigma_M, \sigma_R)$, the unpinched master integrals are

$$\begin{aligned} I^{\boldsymbol{\sigma}}_{abcd} = \int_{-\infty}^{0} & d\tau_L\, d\tau_M\, d\tau_R\, (-\tau_L)^{\nu_{0L}} (-\tau_M)^{\nu_{0M}} (-\tau_R)^{\nu_{0R}} \\ & \times e^{\mathrm{i}(\sigma_L E_1 \tau_L + \sigma_M E_2 \tau_M + \sigma_R E_3 \tau_R)} \\ & \times \mathcal{G}_{\sigma_L\sigma_M,\nu_1;ab}(s_1;\tau_L,\tau_M)\, \mathcal{G}_{\sigma_M\sigma_R,\nu_2;cd}(s_2;\tau_M,\tau_R). \end{aligned} \tag{5.4}$$

Here $a,b,c,d \in \{0,1\}$. Each time integral runs from $-\infty$ to 0 with the Bunch–Davies prescription. The four indices refer to the states at the two vertices of each propagator (L_1, M_1, M_2, R_2), in that order. Thus the Top vector is

$$\boldsymbol{I}^{\boldsymbol{\sigma}}_{\mathrm{Top}} = \big(I^{\boldsymbol{\sigma}}_{0000}, I^{\boldsymbol{\sigma}}_{0001}, \ldots, I^{\boldsymbol{\sigma}}_{1111}\big)^{\mathsf{T}}. \tag{5.5}$$

It contains $2^4 = 16$ masters. In particular, Top 0000 denotes $I^{\boldsymbol{\sigma}}_{0000}$, with no derivative acting on the states, or additional integer shifts in the time powers.

The parameters $\nu_{0L}, \nu_{0M}, \nu_{0R}$ in the integrand are the effective vertex time powers. If p_1, p_2, p_3 are the powers before absorbing the massive mode prefactors, then

$$\begin{aligned} \nu_{0L} &= p_1 + \frac{3}{2} + \nu_1, \qquad \nu_{0R} = p_3 + \frac{3}{2} + \nu_2, \\ \nu_{0M} &= p_2 + 3 + \nu_1 + \nu_2. \end{aligned} \tag{5.6}$$

The vertex factors $\mathrm{i}\sigma_v$ and the external and mode normalisations are not included in (5.4); they are restored when constructing the physical correlator.

In the IBP reduction, derivatives of the functions that impose time ordering generate contact terms that merge adjacent vertices. These terms define the LeftPinch, RightPinch and DoublePinch sectors. Their masters can be labelled $I^{\boldsymbol{\sigma},L}_{cd}$, $I^{\boldsymbol{\sigma},R}_{ab}$ and $I^{\boldsymbol{\sigma},D}$, respectively. The left pinch merges L and M and leaves the two indices of propagator 2; the right pinch merges M and R and leaves those of propagator 1. The double pinch merges all three vertices and leaves no massive propagator. When both internal propagators join vertices with equal contour signs, the full hierarchy is

$$\mathrm{Top}(16) \oplus \mathrm{LeftPinch}(4) \oplus \mathrm{RightPinch}(4) \oplus \mathrm{DoublePinch}(1), \qquad \dim = 25. \tag{5.7}$$

For other contour assignments, only the allowed pinched sectors are included.

At real kinematic points, we compute the four assignments $(+++)$, $(++-)$, $(+-+)$ and $(-++)$. After restoring the vertex factors and mode normalisations, each Top 0000 entry gives one contribution to the physical correlator. Reversing all three contour signs gives the complex conjugate contribution. The four computed terms and their conjugates determine $\mathcal{A}_{\mathrm{phys}}$. To match the phase convention above, the application code passes $-E_v$ as `externalLegEnergy`, uses `NuConvention -> "Negative"`, and supplies the corresponding pinch normalisations.

We set $p_1 = p_2 = p_3 = 0$ and express all energies and momenta in units of E_2. With $q = s_1/E_2$, the scan follows

$$\begin{aligned}(E_1, E_2, E_3, s_1, s_2) &= \left(\frac{100q}{99}, 1, \frac{10}{99}, q, \frac{1}{10}\right), \qquad (\nu_1, \nu_2) = (\mathrm{i}, 2\mathrm{i}),\\ (\nu_{0L}, \nu_{0M}, \nu_{0R}) &= \left(\frac{3}{2} + \mathrm{i}, 3 + 3\mathrm{i}, \frac{3}{2} + 2\mathrm{i}\right).\end{aligned} \tag{5.8}$$

As the numerical benchmark, which is independent of the DE construction, we use the multivariable hypergeometric representation of this correlator derived in ref. [25]. It expresses the signal and background contributions as multiple series containing Appell F_2 functions. Their sum gives the complete physical correlator. These contributions are not the IBP sectors in (5.7). For this benchmark, DE transport evolves the full master vector for each contour assignment and then extracts the physical correlator from its $I^{\boldsymbol{\sigma}}_{0000}$ entries. The hypergeometric calculation evaluates the same physical correlator directly at each target point; it does not separately evaluate all components of the master vectors. The reported comparison is therefore between physical correlators, not a componentwise comparison of all masters. The truncation order is selected by comparing adjacent orders. The reported hypergeometric time is the sum of the accepted pointwise evaluation times.

The DE calculation uses 80 decimal digits and local Taylor series truncated at order 80. An independent transport at order 96 provides an internal stability check. We compare batches of $N = 1$, 10, 50 and 100 target points on the slice in (5.8). For $N = 1$, 10 and 50, transport starts at $q = 0.1$ and ends at $q = 10^{-4}$. The single point is the end of the path, corresponding to $x = -\log_{10} q = 4$. The ten points are selected at approximately equal index intervals from the original 50 points, including both ends. The 100 point batch contains 80 points on this path and 20 additional points from $q = 0.105$ to 0.2. The additional points share the boundary at $q = 0.1$ but require transport in the opposite direction.

The hypergeometric times for these four batches are 22.78, 344.62, 1040.95 and 4242.89 s. The corresponding median DE times are 26.71, 48.82, 48.86 and 57.56 s (figure 3). Each DE time includes physical boundary construction, DE generation and the transport call, with the order 96 backend time subtracted. It includes data preparation and Python invocation. All four DE batches were measured three times, with the execution order varied between rounds. We report the median total time and show the full range of the three measurements. The observed ratios of hypergeometric time to DE time are 7.06, 21.31 and 73.71 for ten, 50 and 100 points, respectively. The isolated end of the path is slower with DE transport than with the hypergeometric representation.

The average DE time per output point decreases from 26.71 s to 4.88, 0.98 and 0.58 s across these batches. Boundary data and transport are shared, rather than recomputed independently for each target. Points encountered along the path use saved node values or evaluations of an already constructed local series. In the new tests, each branch used 29 internal nodes for one output point and 33 nodes for ten output points. Thus, additional outputs did not require a proportional increase in transport work.

Within a convergence disk, a shorter step suppresses higher Taylor terms. It can therefore permit a lower truncation order at a fixed accuracy. This is a local property, not a guarantee of lower total cost, since shorter steps can require more segments.

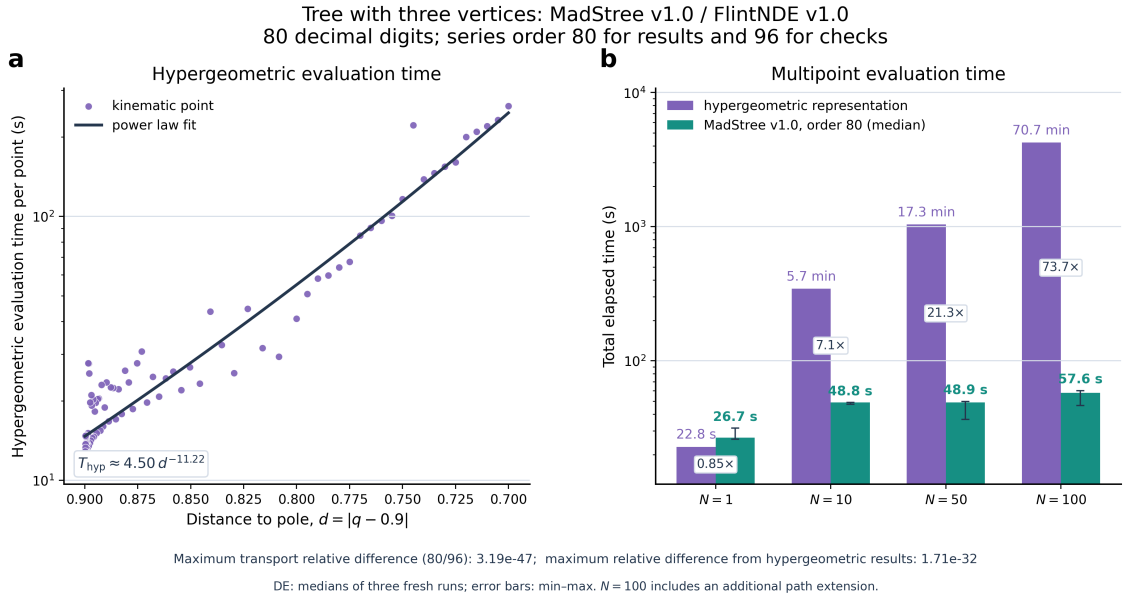


Figure 3. Tree benchmark with MadStree v1.0 and FlintNDE v1.0. **a**, Hypergeometric evaluation time at individual kinematic points. **b**, Total evaluation times for $N = 1$, 10, 50 and 100 target points.

The 100 points in figure 3**a** comprise 80 points with $q = 10^{-x}$ and $x = 1 + 3j/79$ ($j = 0, \ldots, 79$), uniformly spaced in x, and 20 points with $q = 0.1 + 0.005j$ ($j = 1, \ldots, 20$), uniformly spaced in q. Each hypergeometric time uses the accepted truncation order. In panel **b**, the DE bars show medians of three runs, with error bars spanning the minimum and maximum. The times include boundary construction, DE generation and the transport call. The first three batches share the same start and end; the 100 point batch includes the additional path extension. Hypergeometric data are newly computed for one and ten points and retained for 50 and 100 points. The ratio labels give the hypergeometric time divided by the DE time. The figure footer reports the largest relative differences between transport orders and from the hypergeometric results across all four batches.

Across the scan of 100 points, the largest relative difference from the hypergeometric result is 1.712×10^{-32}. The comparison of transport orders 80 and 96 gives a maximum relative difference of 1.308×10^{-50}. At $x = 4$, the physical correlator is

$$\mathcal{A}_{\text{phys}}(4) = 9.87033514798954 \times 10^{-9}, \tag{5.9}$$

with a relative difference of 4.21×10^{-36} from the hypergeometric result.

On the same kinematic configuration, the direct Appell F_2 representation requires $q < 0.89$. The point $q = 0.8905$ lies outside this domain, whereas the nearest positive pole of the DE connection is at $q = 0.9$. To reach $q = 1$, we continue the chosen analytic branch

along

$$0.1 \to 0.86 \to 0.86 + 0.04\mathrm{i} \to 0.93 + 0.04\mathrm{i} \to 0.93 \to 1. \tag{5.10}$$

This contour avoids the pole through the upper half of the complex plane. The continuation records 81 points. The calculation at order 80 takes 35.08 s, or 45.76 s when the reference calculation at order 96 is included. The maximum relative difference between orders 80 and 96 is 1.802×10^{-44}. Within the overlap region, the maximum relative difference from the hypergeometric result is 1.712×10^{-32}. The DE method thus evaluates the selected analytic branch beyond the domain of the direct series representation.

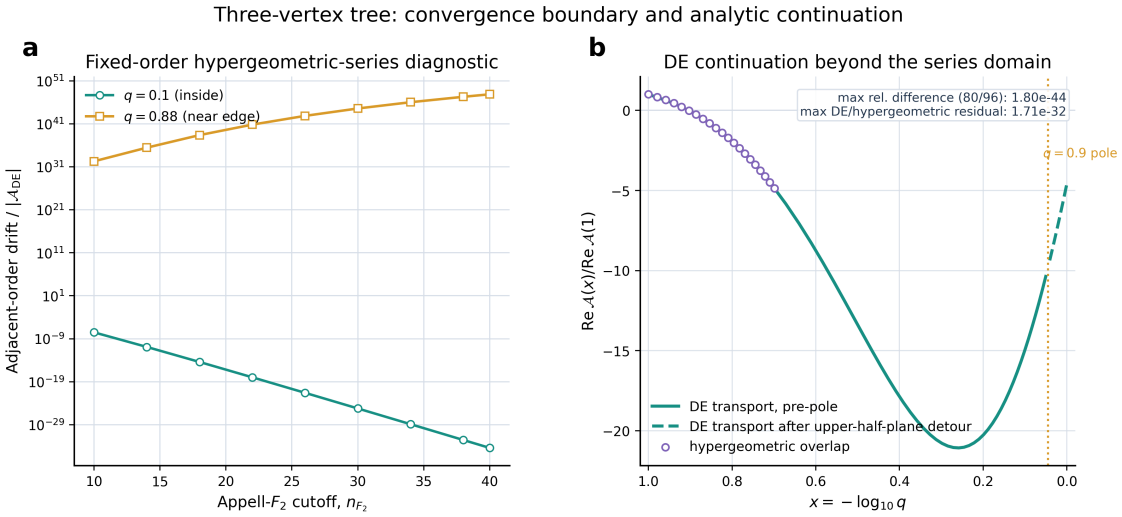


Figure 4. Analytic continuation with MadStree v1.0. **a**, Dependence of the hypergeometric partial sums on the truncation order near the direct convergence boundary. **b**, DE continuation at order 80 along the contour in (5.10).

Bubble time kernel at fixed loop momentum The second example has two time vertices joined by two massive propagators (figure 5). We evaluate the integrals over the two vertex times while holding the internal momenta fixed. We do not integrate over the spatial loop momentum.

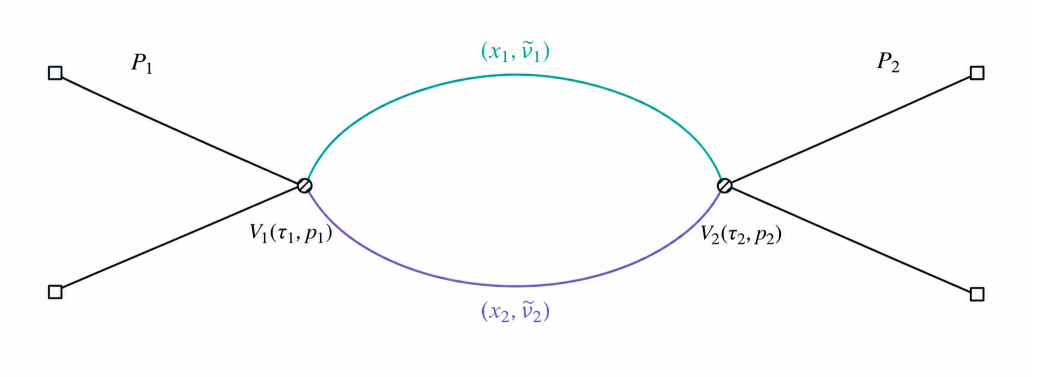


Figure 5. Bubble time kernel with external energy sums P_1, P_2 and fixed internal momentum magnitudes x_1, x_2.

Let τ_1, τ_2 be the vertex times, P_1, P_2 the external energy sums, and x_1, x_2 the internal momentum magnitudes. The Hankel orders of the two internal propagators are ν_{B1}, ν_{B2}. Using the propagator kernels in (5.3), the Top masters are

$$\begin{aligned} J^{\sigma_1\sigma_2}_{abcd} = \int_{-\infty}^{0} d\tau_1\, d\tau_2\, (-\tau_1)^{p_1} (-\tau_2)^{p_2} e^{\mathrm{i}(\sigma_1 P_1 \tau_1 + \sigma_2 P_2 \tau_2)} \\ \times \mathcal{G}_{\sigma_1\sigma_2,\nu_{B1};ab}(x_1; \tau_1, \tau_2) \mathcal{G}_{\sigma_1\sigma_2,\nu_{B2};cd}(x_2; \tau_1, \tau_2). \end{aligned} \tag{5.11}$$

Here $a, b, c, d \in \{0, 1\}$. The indices a, b label the two states that propagator 1 contributes at vertices 1 and 2; c, d label those of propagator 2 in the same vertex order. Here p_1, p_2 are the effective time powers after absorbing the mode prefactors, unlike the bare powers used in (5.6). The contour signs are $\sigma_1, \sigma_2 = \pm 1$. We denote positive and negative signs by p and m, respectively; these labels are unrelated to the time powers p_1, p_2. The four contour assignments are pp, pm, mp and mm. In MadStree v1.0, assignments with equal contour signs have 22 masters after the exact state projection. Assignments with opposite contour signs have only the Top sector, with 16 masters. For each assignment, the projection onto the Top sector is $\boldsymbol{J}_{\sigma_1\sigma_2} = (J^{\sigma_1\sigma_2}_{0000}, J^{\sigma_1\sigma_2}_{0001}, \ldots, J^{\sigma_1\sigma_2}_{1111})^{\mathsf{T}}$. The vertex factors $(\mathrm{i}\sigma_1)(\mathrm{i}\sigma_2)$ give

$$\boldsymbol{K}_{\mathrm{SK}} = -\boldsymbol{J}_{pp} + \boldsymbol{J}_{pm} + \boldsymbol{J}_{mp} - \boldsymbol{J}_{mm}. \tag{5.12}$$

We use $k_s = x_1$ as the momentum unit and vary the dimensionless energy $\rho = P_1/k_s = P_2/k_s$. The parameter values in (5.11) are

$$\begin{aligned} (k_s, x_1, x_2) &= (1, 1, \sqrt{2}), \qquad P_1 = P_2 = \rho, \\ (\nu_{B1}, \nu_{B2}) &= (\mathrm{i}/4, \mathrm{i}/4), \qquad p_1 = p_2 = 1 + \mathrm{i}/2. \end{aligned} \tag{5.13}$$

The normalised Top 0000 component is

$$\widehat{\mathcal{F}}_{\mathrm{i}/4}(\rho) = \frac{[\boldsymbol{K}_{\mathrm{SK}}(\rho)]_{0000}}{[\boldsymbol{K}_{\mathrm{SK}}(10)]_{0000}}. \tag{5.14}$$

The four zeros select the four undifferentiated states of the two massive propagators.

The scan contains 201 points over $2.8 \leq \rho \leq 10$. Calculations use decimal arithmetic with 80 digits. The transport orders are 80 and 96, and the boundary series orders are 17 and 15. The calculation time excluding reference transport at order 96 is 678.20 s. It includes all four contour assignments, DE construction and both boundary calculations. The reference transport takes 5.24 s, and the total recorded time is 683.43 s. The average cost for the scan, including setup, is 3.37 s per point. This average is not the cost of a calculation at a single point.

The maximum relative difference between boundary orders 17 and 15 is 4.740×10^{-24}. The maximum relative difference between transport orders 80 and 96 is 1.025×10^{-50}. These tests assess boundary and transport stability separately; the comparison of transport orders alone is not a bound on the total error. The largest relative difference from the independent scan is 3.70×10^{-14}.

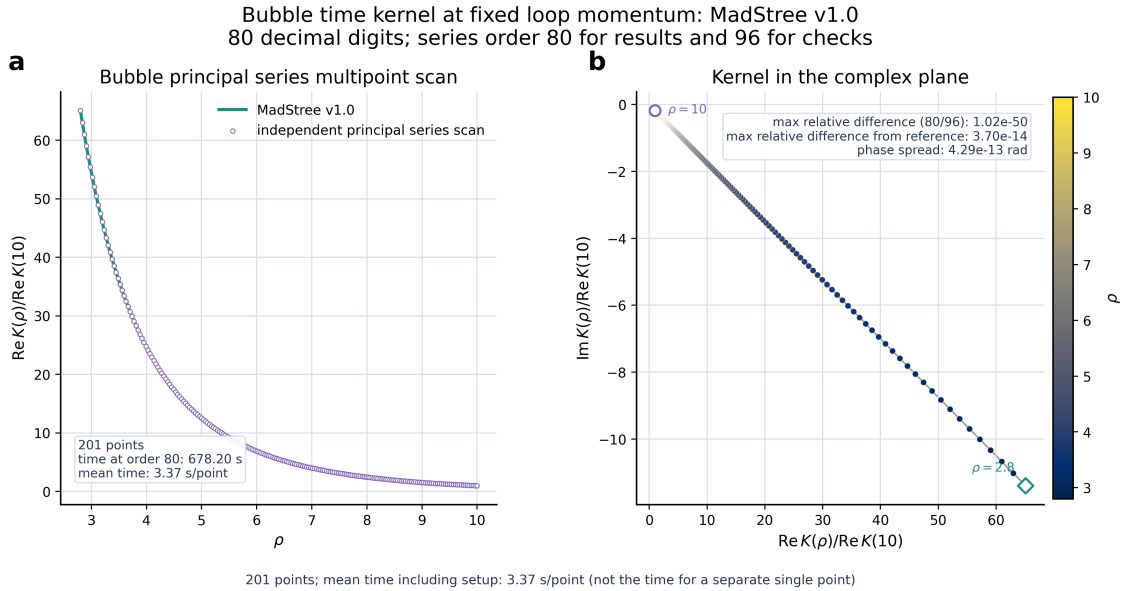


Figure 6. Bubble time kernel evaluated with MadStree v1.0. **a**, The normalised kernel and the independent reference. **b**, The kernel in the complex plane, with colour indicating ρ.

Both panels in figure 6 use the same 201 points. The nearly radial distribution in panel **b** indicates an approximately constant phase. The maximum relative difference between transport orders 80 and 96 is 1.025×10^{-50}. The average calculation time, including setup, is 3.37 s per point; this is the batch average, not an isolated point timing.

The tree example illustrates two uses of DE transport: efficient multipoint evaluation and continuation beyond the convergence domain of a direct series. The bubble example applies the same framework for time integrals to a graph containing a loop at fixed internal momenta.

6 Summary and outlook

`MadStree` implements the general IBP and d log-form construction of [39, 57] as a single computation that starts from the diagram the user supplies and ends at kinematic points, with no intermediate formula written by hand. Three local blocks — the theta-free vertex exponential, the massive two-state block, and the massless block whose four states compress into one shared two-state factor — feed one registry, and that registry alone gives the iterative reduction, the uniform d log-form differential equation, and the contact terms that merge vertices under a common theta function. The boundary data come from the same objects: a strict ranking of the vertex energies freezes every theta function to 0 or 1, the factorized time integrals then return the leading data of all masters as products of Gamma functions, one per time component, and a vertex without an external leg is handled by a private auxiliary energy inside the same system rather than by a separate limit. Because neither the reduction nor the boundary needs the analytic form of the answer, one transport covers a whole path. The tree application of Sec. 5.2 shows the cost per output point falling as points are added to a batch, and shows the same machinery continuing a chosen branch beyond the convergence domain of the direct multivariable series representation.

Two decisions in the implementation matter as much as the formulas. Everything that could invalidate a result is settled before the numerics: the certification carried by `dlogStatus`, the boundary chart certificate without which `MSBoundaryData` refuses to proceed, and the support certificate that fixes the lowest regulator power $n_{\min}$ from the symbolic boundary and the DE, so that the $\varepsilon \to 0$ limit is a Laurent reconstruction and not a fit. When a check fails, the run fails closed and returns a `Failure` object with the captured backend output instead of a silently degraded number. The price of that discipline is a short list of obligations that stay with the user: after a jump across a supported singularity the branch reached by the transport has to be identified by the user, and the comparisons between boundary orders and between transport orders reported in Sec. 5.2 are stability checks, not a bound on the total error.

The scope is the one announced in the abstract: the package computes vertex time integrals, for a tree, or for a loop graph with its internal momenta held fixed, and it does not perform the momentum integrations. That is also the direction of the next step. The vertex blocks, the iterative reduction and the uniform d log-form DE derived here are the kernel of the parity-split IBP system and the d log-form DE of loop integrals in de Sitter space constructed in [58], so the algebraic layers have a loop-level counterpart already written down, while the momentum integrations and the sectors they bring are the part that still has to be built.

A Common-theta bundles and the odd-subset expansion

Bundles and their theta argument. Fix a sector s and write $\mathrm{comp}_s(v)$ for the current component that contains the vertex v; a component is a maximal set of vertices already merged by contacts, and a lone vertex is one component. An uncontracted full propagator

$e = (u_e, v_e)$ carries the theta $\theta(\tau_{u_e} - \tau_{v_e})$ of its two component times, so two propagators e, e' carry the *same* theta if and only if their component pairs coincide as ordered pairs,

$$\begin{gathered}\big(\mathrm{comp}_s(u_e), \mathrm{comp}_s(v_e)\big) = \big(\mathrm{comp}_s(u_{e'}), \mathrm{comp}_s(v_{e'})\big) = (U, V), \\ \text{so} \quad \Delta_e = \Delta_{e'} \equiv \Delta = \tau_U - \tau_V .\end{gathered} \tag{A.1}$$

A maximal set of propagators with this property is the common-theta bundle $\mathcal{B}$ of the pair (U, V), with $|\mathcal{B}| = n \geq 2$; the case $n = 1$ is an ordinary single-propagator contact and is contained in Sec. 3.4. Propagators whose theta arguments differ belong to different bundles, and the bundles are independent because their Δ's are independent variables: the contact events are therefore performed bundle by bundle, never across bundles. The vertex exponentials of Block (i) are theta-free and take no part in this bookkeeping, so only the two-state factors of (3.3) enter the steps below.

Step 1: multiplying the thetas out. For each propagator of one bundle, write the SK pair of products as in (3.3), α_e for the coefficient of $\theta(\Delta)$ and β_e for that of $\theta(-\Delta)$. At the level of the h blocks the two are the two orderings of the Hankel branches,

$$\alpha_e = h^{(1)}_{u_e}\, h^{(2)}_{v_e}, \qquad \beta_e = h^{(2)}_{u_e}\, h^{(1)}_{v_e}, \tag{A.2}$$

where the orders, states and arguments are those of (3.8); G_{--} interchanges the two names, and this is the only place the sign σ_e of the bundle enters. The product of the bundle is a product of n binomials in the two functions $\theta(\Delta)$ and $\theta(-\Delta)$. Expanding naively gives 2^n terms, but inside one bundle $\theta(\Delta)$ is the *same* function for every factor, so idempotency $\theta(\Delta)^2 = \theta(\Delta)$ makes all powers of one argument collapse to that argument, and orthogonality $\theta(\Delta)\theta(-\Delta) = 0$ annihilates every term containing both arguments. Exactly the two terms of (3.58) survive. The factor 2^n thus never appears within a bundle: it counts the events of n propagators with pairwise different theta arguments, which is the case of n different bundles, and those are enumerated bundle by bundle.

Step 2: one theta gives one delta. Differentiating the two-term form with respect to the single variable Δ gives

$$\partial_\Delta \left[\theta(\Delta) \prod_{e\in\mathcal{B}} \alpha_e + \theta(-\Delta) \prod_{e\in\mathcal{B}} \beta_e \right] = \delta(\Delta) \left(\prod_{e\in\mathcal{B}} \alpha_e - \prod_{e\in\mathcal{B}} \beta_e \right), \tag{A.3}$$

because $\partial_\Delta \theta(\Delta) = \delta(\Delta)$ and $\partial_\Delta \theta(-\Delta) = -\delta(\Delta)$. Three consequences are worth stating explicitly. First, the bundle produces *one* distribution, not n of them: the whole contact information of the bundle is concentrated in the single coefficient $\prod_e \alpha_e - \prod_e \beta_e$, which is antisymmetric under $\alpha \leftrightarrow \beta$. Second, that one $\delta(\Delta)$ forces the two component times to coincide *once*, so the components U and V merge once, independently of how many factors d_e a term of the coefficient contains; this is what fixes the merge count in (3.36). Third, for G_{--} the interchange of α and β turns (A.3) into its negative, which reproduces the single sign σ_e of (3.24) for a whole bundle.

Step 3: telescoping the antisymmetric difference. Order the propagators of the bundle as $1, \dots, n$ and abbreviate $\alpha_e = A_e$, $\beta_e = B_e$. Inserting and subtracting successive partial products gives the telescoping identity

$$\begin{aligned} \prod_{e=1}^{n} A_e - \prod_{e=1}^{n} B_e &= \sum_{k=1}^{n} \left(\prod_{e \le k} A_e \prod_{e > k} B_e - \prod_{e < k} A_e \prod_{e \ge k} B_e \right) \\ &= \sum_{k=1}^{n} D_k \prod_{e<k} A_e \prod_{e>k} B_e, \end{aligned} \tag{A.4}$$

whose intermediate sum collapses because the k-th second summand and the $(k{+}1)$-th first summand are identical, while the first summand of $k = 1$ and the last summand of $k = n$ supply the two products of (A.3). Each term of (A.4) contains exactly one difference factor $D_e = A_e - B_e$.

Step 4: collecting by subsets and the parity. Introduce, as in (3.59), the difference and the mean of the two products, $\mathsf{d}_e = D_e = A_e - B_e$ and $\mathsf{m}_e = (A_e + B_e)/2$, equivalently $A_e = \mathsf{m}_e + \mathsf{d}_e/2$ and $B_e = \mathsf{m}_e - \mathsf{d}_e/2$, and expand the spectators of (A.4) in this basis. In the k-th term the propagator k supplies the explicit d_k; a propagator $e \in S$ with $e < k$ contributes $+\mathsf{d}_e/2$, a propagator $e \in S$ with $e > k$ contributes $-\mathsf{d}_e/2$, and every propagator outside S contributes m_e. So the coefficient of the monomial $\prod_{e \in S} \mathsf{d}_e \prod_{e \notin S} \mathsf{m}_e$ is

$$c_S = 2^{1-|S|} \sum_{k \in S} (-1)^{\#\{e \in S \colon e > k\}}. \tag{A.5}$$

Writing $S = \{i_1 < \cdots < i_m\}$ with $m = |S|$, the exponent for $k = i_j$ is $m - j$, hence

$$\sum_{j=1}^{m} (-1)^{m-j} = \sum_{r=0}^{m-1} (-1)^r = \begin{cases} 1, & m \text{ odd}, \\ 0, & m \text{ even}, \end{cases} \qquad c_S = \big(1 - (-1)^{|S|}\big) 2^{-|S|}, \tag{A.6}$$

which is (3.59): only odd subsets survive, each with the pure numerical weight $2^{1-|S|}$. The empty set cannot occur, since every telescoping term carries at least the explicit d_k, in agreement with $c_\varnothing = 0$. The same parity follows from symmetry without any enumeration: the left-hand side of (A.4) changes sign under $\alpha \leftrightarrow \beta$, in which d_e is odd and m_e even, so a monomial with an even number of d factors must cancel; the explicit computation (A.5) only fixes the remaining coefficients.

Events, weights and the bookkeeping. Each non-empty odd subset S of one bundle is one simultaneous contact event. Its injection matrix $C_{st}^{(v,1)}$ of (3.45) is the tensor product of the local contact selectors of the propagators in S — $|1\rangle_e$ on a massless shared slot, the complementary-bit selector of (3.44) on a pair of massive-propagator states — multiplied by $2^{1-|S|}$ and by the spectator factors m_e of the unselected bundle propagators, then embedded in the global bit order with all spectator bits of other propagators preserved. The collected form (3.59), not the term-by-term form (A.4), is what the program enumerates: the k-th telescoping term stands for "contract only the k-th propagator," but after the spectators

are expanded the same subset S is reached from several k, so an enumeration over k would generate the same sector transition more than once. After collecting, each event appears exactly once with a numerical weight, and the deduplication key of the reduction is the triple (parent sector, child sector, set of selected propagator identifiers), which is finite because every event strictly reduces the number of current components.

Child base power and coincident unselected propagators. For a component C produced by such events the base power is (3.36), $A_{C,s} = \sum_{v\in C} \nu_{0,v} + \sum_{e\in E_s(C)} \varrho_e + |C| - 1$. The three terms have three different origins and should not be conflated: the vertex powers add because the merged integrand is a product, each contracted propagator adds its own raw contact power ϱ_e because the corresponding δ brings one factor $(-\tau)^{\varrho_e}$, and the last term counts *merges of components*, which is $|C| - 1$ for any component of $|C|$ vertices and does not grow with the number of bundle propagators contracted at once. Since (3.36) depends only on the final component and on the contracted set, and not on the order or the clustering of the events, different legal event sequences that reach the same sector give the same child base power; the reduction is confluent at the level of the powers.

An unselected propagator whose two vertices fall into one component after a bundle event must also be reduced at equal times. For a massless full propagator the shared slot then contributes only its symmetric state, because the two Wightman combinations coincide at coincident times, so the odd shared state of the propagator gives zero; for a massive full propagator the two states carrying a single derivative become identified, $10 \sim 01$, which is the equal-time canonical quotient. Both cases are implemented the same way: the matrices are first built in the raw binary space of all massive and shared slots and then retracted to the true sector masters by the constant maps of (3.21), $P_s\, M^{\mathrm{raw}}\, S_s$, so the equal-time quotients require no further computation and no dense inversion.

Relation to the single-contact case. For $|\mathcal{B}| = 1$ the two-term form (3.58) reduces to the ordinary SK pair of one propagator, (A.4) to the single difference d_e, and $c_{\{e\}} = 1$, so the bundle machinery reproduces exactly the single-propagator contacts of Sec. 3.4 and of [57], where every contact is attached to one full propagator. The new ingredients are the weight $2^{1-|S|}$, the parity selection, and the identification of the merge count with $|C| - 1$ rather than with the number of contracted propagators.

References

[1] S. Weinberg, *Quantum contributions to cosmological correlations*, *Phys. Rev. D* **72** (2005) 043514 [hep-th/0506236].

[2] X. Chen, Y. Wang and Z.-Z. Xianyu, *Schwinger-Keldysh Diagrammatics for Primordial Perturbations*, *JCAP* **12** (2017) 006 [1703.10166].

[3] N. Arkani-Hamed, D. Baumann, H. Lee and G.L. Pimentel, *The Cosmological Bootstrap: Inflationary Correlators from Symmetries and Singularities*, *JHEP* **04** (2020) 105 [1811.00024].

[4] D. Baumann, C. Duaso Pueyo, A. Joyce, H. Lee and G.L. Pimentel, *The cosmological bootstrap: weight-shifting operators and scalar seeds*, *JHEP* **12** (2020) 204 [1910.14051].

[5] D. Baumann, C. Duaso Pueyo, A. Joyce, H. Lee and G.L. Pimentel, *The Cosmological Bootstrap: Spinning Correlators from Symmetries and Factorization*, *SciPost Phys.* **11** (2021) 071 [2005.04234].

[6] E. Pajer, D. Stefanyszyn and J. Supeł, *The Boostless Bootstrap: Amplitudes without Lorentz boosts*, *JHEP* **12** (2020) 198 [2007.00027].

[7] E. Pajer, *Building a Boostless Bootstrap for the Bispectrum*, *JCAP* **01** (2021) 023 [2010.12818].

[8] G. Cabass, E. Pajer, D. Stefanyszyn and J. Supeł, *Bootstrapping large graviton non-Gaussianities*, *JHEP* **05** (2022) 077 [2109.10189].

[9] D. Baumann, D. Green, A. Joyce, E. Pajer, G.L. Pimentel, C. Sleight et al., *Snowmass White Paper: The Cosmological Bootstrap*, *SciPost Phys. Comm. Rep.* **2024** (2024) 1 [2203.08121].

[10] G.L. Pimentel and D.-G. Wang, *Boostless cosmological collider bootstrap*, *JHEP* **10** (2022) 177 [2205.00013].

[11] S. Jazayeri and S. Renaux-Petel, *Cosmological bootstrap in slow motion*, *JHEP* **12** (2022) 137 [2205.10340].

[12] Z. Qin and Z.-Z. Xianyu, *Helical inflation correlators: partial Mellin-Barnes and bootstrap equations*, *JHEP* **04** (2023) 059 [2208.13790].

[13] D.-G. Wang, G.L. Pimentel and A. Achúcarro, *Bootstrapping multi-field inflation: non-Gaussianities from light scalars revisited*, *JCAP* **05** (2023) 043 [2212.14035].

[14] Z. Qin and Z.-Z. Xianyu, *Closed-form formulae for inflation correlators*, *JHEP* **07** (2023) 001 [2301.07047].

[15] S. Aoki, L. Pinol, F. Sano, M. Yamaguchi and Y. Zhu, *Cosmological correlators with double massive exchanges: bootstrap equation and phenomenology*, *JHEP* **09** (2024) 176 [2404.09547].

[16] H. Liu and Z.-Z. Xianyu, *Massive inflationary amplitudes: differential equations and complete solutions for general trees*, *JHEP* **09** (2025) 183 [2412.07843].

[17] Z. Qin, S. Renaux-Petel, X. Tong, D. Werth and Y. Zhu, *The exact and approximate tales of boost-breaking cosmological correlators*, *JCAP* **09** (2025) 058 [2506.01555].

[18] Z.-Z. Xianyu and J. Zang, *Massive inflationary amplitudes: new representations and degenerate limits*, *JHEP* **03** (2026) 122 [2511.08677].

[19] C. Sleight, *A Mellin Space Approach to Cosmological Correlators*, *JHEP* **01** (2020) 090 [1906.12302].

[20] C. Sleight and M. Taronna, *Bootstrapping Inflationary Correlators in Mellin Space*, *JHEP* **02** (2020) 098 [1907.01143].

[21] C. Sleight and M. Taronna, *From AdS to dS exchanges: Spectral representation, Mellin amplitudes, and crossing*, *Phys. Rev. D* **104** (2021) L081902 [2007.09993].

[22] C. Sleight and M. Taronna, *From dS to AdS and back*, *JHEP* **12** (2021) 074 [2109.02725].

[23] A. Premkumar, *Regulating loops in de Sitter spacetime*, *Phys. Rev. D* **109** (2024) 045003 [2110.12504].

[24] Z. Qin and Z.-Z. Xianyu, *Phase information in cosmological collider signals*, *JHEP* **10** (2022) 192 [2205.01692].

[25] Z.-Z. Xianyu and J. Zang, *Inflation correlators with multiple massive exchanges*, *JHEP* **03** (2024) 070 [2309.10849].

[26] Z. Qin, *Cosmological correlators at the loop level*, *JHEP* **03** (2025) 051 [2411.13636].

[27] D. Marolf and I.A. Morrison, *The IR stability of de Sitter: Loop corrections to scalar propagators*, *Phys. Rev. D* **82** (2010) 105032 [1006.0035].

[28] Z.-Z. Xianyu and H. Zhang, *Bootstrapping one-loop inflation correlators with the spectral decomposition*, *JHEP* **04** (2023) 103 [2211.03810].

[29] M. Loparco, J. Penedones, K. Salehi Vaziri and Z. Sun, *The Källén-Lehmann representation in de Sitter spacetime*, *JHEP* **12** (2023) 159 [2306.00090].

[30] D. Werth, *Spectral representation of cosmological correlators*, *JHEP* **12** (2024) 017 [2409.02072].

[31] H. Zhang, *Dimensional regularization of bubble diagrams in de Sitter spacetime*, *JHEP* **02** (2026) 119 [2507.19318].

[32] B.L. Altshuler, *Kallen-Lehmann representation for Spinor-Scalar Loops in de Sitter Space-time. Spectral equations*, *JHEP* **12** (2025) 102 [2508.07467].

[33] H. Liu, Z. Qin and Z.-Z. Xianyu, *Dispersive bootstrap of massive inflation correlators*, *JHEP* **02** (2025) 101 [2407.12299].

[34] S. Das, D. Karan, B. Khatun and N. Kundu, *Reconstructing cosmological correlators via dispersion: from cutting to dressing rules*, 2602.05546.

[35] L.-T. Wang, Z.-Z. Xianyu and Y.-M. Zhong, *Precision calculation of inflation correlators at one loop*, *JHEP* **02** (2022) 085 [2109.14635].

[36] D. Werth, L. Pinol and S. Renaux-Petel, *Cosmological Flow of Primordial Correlators*, *Phys. Rev. Lett.* **133** (2024) 141002 [2302.00655].

[37] L. Pinol, S. Renaux-Petel and D. Werth, *The cosmological flow: a systematic approach to primordial correlators*, *JCAP* **02** (2025) 019 [2312.06559].

[38] D. Werth, L. Pinol and S. Renaux-Petel, *CosmoFlow: Python Package for Cosmological Correlators*, *Class. Quant. Grav.* **41** (2024) 175015 [2402.03693].

[39] J. Chen, B. Feng and Y.-X. Tao, *Multivariate hypergeometric solutions of cosmological (dS) correlators by d log-form differential equations*, *JHEP* **03** (2025) 075 [2411.03088].

[40] K.G. Chetyrkin and F.V. Tkachov, *Integration by parts: The algorithm to calculate β-functions in 4 loops*, *Nucl. Phys. B* **192** (1981) 159.

[41] A.V. Kotikov, *Differential equations method: New technique for massive Feynman diagrams calculation*, *Phys. Lett. B* **254** (1991) 158.

[42] A.V. Kotikov, *Differential equation method: The Calculation of N point Feynman diagrams*, *Phys. Lett. B* **267** (1991) 123.

[43] Z. Bern, L.J. Dixon and D.A. Kosower, *Dimensionally regulated pentagon integrals*, *Nucl. Phys. B* **412** (1994) 751 [hep-ph/9306240].

[44] T. Gehrmann and E. Remiddi, *Differential equations for two-loop four-point functions*, *Nucl. Phys. B* **580** (2000) 485 [hep-ph/9912329].

[45] J.M. Henn, *Multiloop integrals in dimensional regularization made simple*, *Phys. Rev. Lett.* **110** (2013) 251601 [1304.1806].

[46] A.V. Smirnov and F.S. Chukharev, *FIRE6: Feynman Integral REduction with modular arithmetic*, *Comput. Phys. Commun.* **247** (2020) 106877 [1901.07808].

[47] A.V. Smirnov and M. Zeng, *FIRE 7: Automatic Reduction with Modular Approach*, *Comput. Phys. Commun.* **325** (2026) 110200 [2510.07150].

[48] P. Maierhöfer, J. Usovitsch and P. Uwer, *Kira—A Feynman integral reduction program*, *Comput. Phys. Commun.* **230** (2018) 99 [1705.05610].

[49] J. Klappert, F. Lange, P. Maierhöfer and J. Usovitsch, *Integral reduction with Kira 2.0 and finite field methods*, *Comput. Phys. Commun.* **266** (2021) 108024 [2008.06494].

[50] F. Lange, J. Usovitsch and Z. Wu, *Kira 3: integral reduction with efficient seeding and optimized equation selection*, *Comput. Phys. Commun.* **322** (2026) 109999 [2505.20197].

[51] M. Hidding, *DiffExp, a Mathematica package for computing Feynman integrals in terms of one-dimensional series expansions*, *Comput. Phys. Commun.* **269** (2021) 108125 [2006.05510].

[52] X. Liu, Y.-Q. Ma and C.-Y. Wang, *A Systematic and Efficient Method to Compute Multi-loop Master Integrals*, *Phys. Lett. B* **779** (2018) 353 [1711.09572].

[53] X. Liu, Y.-Q. Ma, W. Tao and P. Zhang, *Calculation of Feynman loop integration and phase-space integration via auxiliary mass flow*, *Chin. Phys. C* **45** (2021) 013115 [2009.07987].

[54] X. Liu and Y.-Q. Ma, *AMFlow: A Mathematica package for Feynman integrals computation via auxiliary mass flow*, *Comput. Phys. Commun.* **283** (2023) 108565 [2201.11669].

[55] R.-J. Huang, X. Liu and Y.-Q. Ma, *AMFlow 2.0: significant algorithmic and software improvements for Feynman integral evaluation*, 2607.08477.

[56] Z.-F. Liu and Y.-Q. Ma, *Determining Feynman Integrals with Only Input from Linear Algebra*, *Phys. Rev. Lett.* **129** (2022) 222001 [2201.11637].

[57] J. Chen and B. Feng, *Towards systematic evaluation of de Sitter correlators via Generalized Integration-By-Parts relations*, *JHEP* **06** (2024) 199 [2401.00129].

[58] J. Chen, B. Feng, Z. Qin and Y.-X. Tao, *Loop integrals in de Sitter spacetime: The parity-split IBP system and* d log*-form differential equations*, 2604.14549.

[59] "dSIBP: a package for generating IBP seeds for QFT in de Sitter spacetime."